
Learning GUI Grounding with Spatial Reasoning from Visual Feedback

Yu Zhao¹ Wei-Ning Chen² Huseyin A. Inan² Samuel Kessler² Lu Wang² Lukas Wutschitz² Fangkai Yang²
Chaoyun Zhang² Pasquale Minervini^{1 3} Saravan Rajmohan² Robert Sim²

Abstract

Graphical User Interface (GUI) grounding is commonly framed as a coordinate prediction task – given a natural language instruction, generate on-screen coordinates for actions such as clicks and keystrokes. However, recent Vision Language Models (VLMs) often fail to predict accurate numeric coordinates when processing GUI images with high resolutions and complex layouts. To address this issue, we reframe GUI grounding as an interactive search task, where the VLM generates actions to move a cursor in the GUI to locate UI elements. At each step, the model determines the target object, evaluates the spatial relations between the cursor and the target, and moves the cursor closer to the target conditioned on the movement history. In this interactive process, the rendered cursor provides visual feedback to help the model align its predictions with the corresponding on-screen locations. We train our GUI grounding model, GUI-CURSOR, using multi-step online reinforcement learning with a dense trajectory-based reward function. Experimental results demonstrate that GUI-CURSOR surpasses strong baselines in GUI grounding and agentic tasks, achieving superior performance with the same base models while requiring less training data. Further analysis shows that GUI-CURSOR learns to adaptively conduct more steps on more difficult examples, and it obtains better spatial reasoning capability on out-of-distribution domains.

1. Introduction

Graphical User Interface (GUI) agents solve tasks by translating user instructions into multiple GUI interaction steps (Deng et al., 2023; Li et al., 2024; Zhang et al., 2025a).

¹University of Edinburgh, UK ²Microsoft ³Miniml.AI. Correspondence to: Yu Zhao <yu.zhao@ed.ac.uk>.

A core component of these agents is GUI *grounding*, i.e., identifying a specific pixel coordinate to execute an action, like a click or keystroke. While early approaches rely on extracting coordinates from textual representations of GUIs (e.g., HTML and DOM trees), such data is often redundant and is not consistently available across platforms (Zhang et al., 2025c;b; Wu et al., 2025b). Thus, recent research emphasises pure vision-driven GUI grounding that perceives the interface directly from rendered screenshots (Xu et al., 2024; Gou et al., 2025; Yuan et al., 2025).

However, accurately predicting the precise numerical coordinates of an element on a high-resolution GUI image remains a challenge for current vision-language models (VLMs). This difficulty largely arises from the problem of *spatial semantic alignment* (Wu et al., 2025b), which requires a language model to generate discrete coordinate tokens by implicitly mapping its understanding of a visual element to a specific set of coordinates on the GUI image. Current methods, whether based on supervised fine-tuning (Gou et al., 2025; Wu et al., 2024; Xu et al., 2024), reinforcement learning (Yuan et al., 2025; Liu et al., 2025c; Yang et al., 2025a), or architectural modifications (Wu et al., 2025b), commonly frame GUI grounding as a *one-step coordinate prediction problem during training*: given a screenshot, the VLM is tasked with generating the coordinates of the target. One potential limitation of this one-step learning paradigm is the *absence of a visual feedback loop*: the model is only supervised on the generated numerical coordinates, but does not receive any information on where its prediction actually lands on the GUIs. As a consequence, the model may fail to develop a robust alignment between its numerical predictions and their corresponding on-screen positions.

To address this limitation, we propose GUI-CURSOR, which reframes GUI grounding as a search task in an interactive environment (Section 2.2), as shown in Figure 1. During training, GUI-CURSOR operates by controlling a virtual cursor on a GUI image; at each step, the model performs spatial reasoning to determine if the cursor is positioned over the target GUI element. If the model determines that the cursor is not positioned over the target, it predicts a new position, and the environment provides an updated image of the GUI with the cursor located in the new position. This iterative process terminates when the model determines the cursor

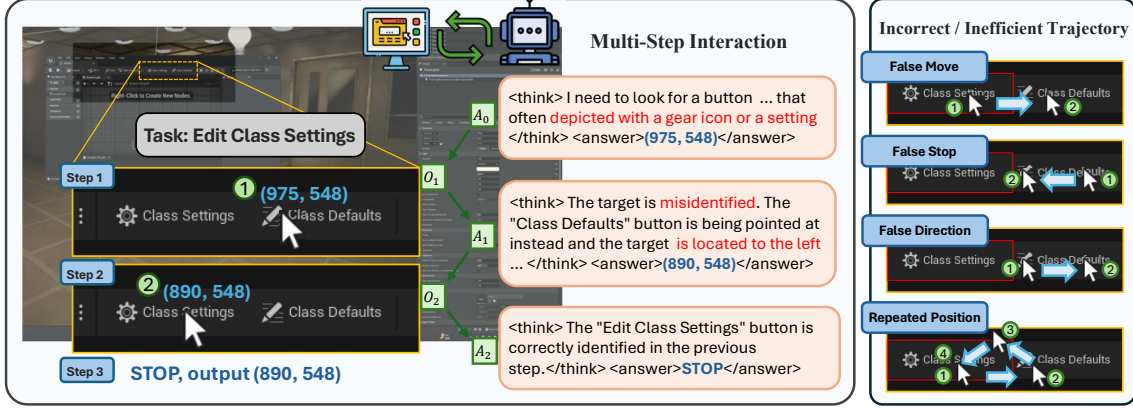


Figure 1. The workflow of our method GUI-CURSOR. The left part presents an example where GUI-CURSOR locates the target successfully: at the first step, the model correctly describes the shape of the target and predicts a coordinate; at the second step, the model evaluates the spatial relationship between the cursor and the target, and it finds the cursor is not correctly positioned, then provides a new position; at the third step, the model terminates the interaction. The right part presents the examples of incorrect or inefficient moving trajectories penalised in our reward function.

is correctly positioned on the target UI element, or when a maximum number of cursor moving steps is reached. This interactive process improves GUI grounding training and inference in two ways: 1) the rendered cursor provides *explicit visual feedback* that shows the predicted position on the GUI image, helping the model better align numerical coordinates with their corresponding GUI elements during training; and 2) this interactive environment allows the model to conduct spatial reasoning based on its visual feedback, allowing it to incrementally refine its predictions at inference time.

We train the model using reinforcement learning (RL) in this interactive environment, as it provides a natural framework for this sequential decision-making task. We design a reward function that combines a position-based reward with several trajectory penalties that discourage wrong or inefficient search processes (Section 2.3). Our ablation studies (Section 3.2) show that these penalties are helpful to prevent degenerate behaviour and significantly improve the downstream accuracy of the model. Furthermore, to improve the efficiency of this interactive grounding process, we introduce a strategy that balances training cost and inference accuracy (Section 2.4).

Experimental results on GUI grounding and agentic benchmarks show that GUI-CURSOR outperforms all strong baseline models consistently based on Qwen2.5-VL-7B (Bai et al., 2025) and UI-TARS-1.5-7B (Qin et al., 2025). When using the same base model, GUI-CURSOR outperforms prior leading model GTA1 (Yang et al., 2025a) by 8.0% on ScreenSpot-Pro (Li et al., 2025b) while only requiring 8K training samples compared to 64K samples used by GTA1. In terms of computational efficiency, the fine-tuned model learns to solve the problem within two steps for 95% of the instances and adaptively conducts more steps on difficult

tasks, such as locating UI elements with small sizes.

To test the spatial reasoning capability of the model, we design a cursor-in-box test – asking the model to classify the spatial relationship between a cursor and a box in a clean background (Section 3.3). We observe that even the strong base model struggles with this simple task, which suggests its grounding ability may not be founded on a robust spatial understanding of images, while GUI-CURSOR obtains higher accuracy without being explicitly trained on it. Further evaluation on spatial reasoning benchmarks indicates that learning GUI grounding by moving a cursor improves the spatial reasoning capability that can generalise to out-of-distribution domains.

Our work makes the following contributions: 1) we reformulate GUI grounding from the static, one-step task to a dynamic, interactive process, which enables the model to learn coordinate-spatial alignment from visual feedback during training; 2) we introduce GUI-CURSOR, the GUI grounding model trained with reinforcement learning in the interactive environment, using a dense, trajectory-based reward function; 3) our comprehensive evaluation across GUI grounding, online agentic, and spatial reasoning benchmarks shows that learning GUI grounding by moving a cursor improves both grounding accuracy and spatial reasoning capability.

2. GUI Grounding with Visual Feedback

We introduce GUI-CURSOR, a method that learns GUI grounding by moving a virtual cursor in an interactive environment. This interactive environment provides explicit visual feedback through the cursor to help the model better align the coordinate and its actual spatial position on the image. During inference, it also allows the model to refine

the prediction according to the spatial relation of the target and the cursor. We train this iterative behaviour using reinforcement learning, as it provides a natural framework for optimising this sequential decision-making process. In the following, we introduce the process of GUI grounding by moving a cursor (Section 2.2), reward function modelling (Section 2.3), and a strategy to balance training efficiency and inference accuracy (Section 2.4).

2.1. Problem Formulation

The task of GUI grounding is to map a natural language instruction to a target coordinate on a GUI. Formally, given a GUI screenshot O with $W \times H$ pixels and a natural language instruction I describing a target element, the goal is to predict an integer coordinate pair (x, y) , with $x, y \in \mathbb{N}_0$, where an action should be performed at this pixel. The ground truth is the rectangular area defined by the bounding box B . Given the top-left corner $(x_{\min}, y_{\min})$ and the bottom-right corner $(x_{\max}, y_{\max})$, this area is the set of all points (x, y) such that: $B = \{(x, y) | x_{\min} \leq x \leq x_{\max}, y_{\min} \leq y \leq y_{\max}\}$. The prediction (x, y) is correct when $(x, y) \in B$.

2.2. GUI Grounding by Moving a Cursor

We design an interactive environment that allows the model to locate the GUI element by moving a cursor. The interaction process consists of a sequence of steps. At the initial step $t = 0$, the screenshot O_0 displays with a cursor at its centre (x_0, y_0) ; conditioned on a natural language instruction I and the observation O_0 , the model generates the response A_0 that contains an action to move the cursor to a position (x_1, y_1) . At each subsequent step t , the new observation O_t displays the cursor at the latest position (x_t, y_t) , and the model generates the response A_t conditioned on the interaction history and the new observation: $(I, O_0, A_0, O_1, \dots, A_{t-1}, O_t)$. Each response consists of a thinking process before an action: $A_t = \langle \text{think} \rangle w_1, w_2, \dots, w_n \langle \text{think} \rangle \langle \text{answer} \rangle v_1, v_2, \dots, v_m \langle \text{answer} \rangle$. The thinking tokens $\{w_i\}_{i=1}^n$ contain the analysis about the target UI element, the current cursor’s location, and the spatial relationship between the target and the cursor. The answer tokens $\{v_i\}_{i=1}^m$ can be either a new coordinate $\{v_i\}_{i=1}^m = (x_t, y_t)$, or $\{v_i\}_{i=1}^m = \text{STOP}$ when the model judges the cursor is correctly on the target. If the new coordinate (x_t, y_t) is provided, the cursor will be rendered at the position (x_t, y_t) for the next turn observation O_{t+1} ; otherwise, the position of the cursor will not be updated.

The episode terminates when the model outputs STOP (i.e., $\{v_i\}_{i=1}^m = \text{STOP}$) or a pre-defined maximum number of steps is reached. Then, the final position (x_T, y_T) of the cursor is returned as the grounding prediction, where T is the number of steps. Figure 1 illustrates how the model

locates the target by moving a cursor in this interactive environment.

2.3. Trajectory Reward Modelling

Our reward function aims to guide the model to obtain both an accurate final prediction and a rational search behaviour. To achieve these goals, we introduce 1) a position-based reward to minimise the distance between the final position and the target, and 2) several trajectory penalties to discourage unreasonable search processes. Existing GUI grounding methods that use RL frameworks (Yang et al., 2025a; Yuan et al., 2025) mainly apply a position-based reward. Differently, regulating the search process is important in our multi-step interaction approach, because it helps to prevent degenerate strategies, e.g., immediately stopping without using visual feedback or repeatedly moving to a visited position. In the following, we introduce the position reward and trajectory penalties used in GUI-CURSOR.

Position Reward This reward measures the quality of the final cursor position, (x_T, y_T) , relative to the target bounding box B . We adopt the dense distance reward used by SE-GUI (Yuan et al., 2025), which considers both the distance to the box and centrality within it:

$$r_p = \begin{cases} 1 + \left(1 - \frac{d_{\text{centre}}((x_T, y_T), B)}{d_{\text{max}}(B)}\right)^2 & \text{if } (x_T, y_T) \in B \\ 1 - d_{\text{edge}}((x_T, y_T), B) & \text{otherwise,} \end{cases} \quad (1)$$

where $d_{\text{edge}}((x, y), B)$ is the Euclidean distance of (x, y) to the nearest edge of B , $d_{\text{max}}(B)$ is the distance of the vertex of B to the centre of B , and $d_{\text{centre}}((x, y), B)$ is the distance to the centre of B . We normalise all distances by the width and height of the image. In Equation (1), the position reward is $1 - d_{\text{edge}}$ if (x_T, y_T) is outside B , increasing with proximity to the centre when inside B .

Trajectory Penalties While the position reward defines a final goal, it provides no guidance on the search process. To guide the model in learning a rational search strategy, we introduce four trajectory-based penalties that target specific undesirable behaviours. We present the examples of the penalised trajectories in the right part of Figure 1.

False Stop Penalty (r_{FS}): Checks if the model outputs STOP but the final cursor position p_T is outside the target box B :

$$r_{\text{FS}} = \mathbb{I}[A_T = \text{STOP} \wedge (x_T, y_T) \notin B] \quad (2)$$

False Move Penalty (r_{FM}): Checks if there is a history position inside the target box, but the model does not output STOP and the final position p_T is outside of it:

$$r_{\text{FM}} = \mathbb{I}[(\exists t < T \text{ s.t. } (x_t, y_t) \in B) \wedge ((x_T, y_T) \notin B)] \quad (3)$$

False Direction Penalty (r_{FD}): Checks if the final position (x_T, y_T) is further from the target box than the initial pre-

diction (x_1, y_1) was, in which case the movement does not shorten the distance between the prediction and the target:

$$r_{FD} = \mathbb{I}[d_{\text{edge}}((x_T, y_T), B) > d_{\text{edge}}((x_1, y_1), B)] \quad (4)$$

Repeated Position Penalty (r_{RP}): Checks if the model predicts the same coordinate more than once:

$$r_{RP} = \mathbb{I}[\exists i, j \in 1, \dots, T \text{ s.t. } i \neq j \wedge (x_i, y_i) = (x_j, y_j)] \quad (5)$$

Then, we define the trajectory reward R_T as a weighted combination of the position reward r_p and a sum of the trajectory penalties r_{FS} , r_{FM} , r_{FD} , and r_{RP} weighted by an hyper-parameter w_p :

$$R_T = r_p - w_p (r_{FD} + r_{FS} + r_{FM} + r_{RP}) \quad (6)$$

Training Objective We use Group Relative Policy Optimisation (GRPO) to optimise the interaction policy guided by the trajectory reward R_T and a format reward to ensure the valid output format (DeepSeek-AI, 2025). GRPO has been successfully applied in prior RL-based GUI grounding methods (Luo et al., 2025a; Yuan et al., 2025; Yang et al., 2025a), and we also use it for its advantages in training stability and efficiency. More details are presented Appendix A.1.

2.4. Cursor-Centric Focusing

The iterative nature of GUI-CURSOR could be computationally demanding, as it processes a growing sequence of interaction history $(I, O_0, A_0, \dots, A_{t-1}, O_t)$ to generate an action. This becomes infeasible when handling native high-resolution screenshots. To alleviate this issue, we use a two-part strategy that balances computational efficiency with predictive accuracy.

During training, we downscale the large image to a manageable resolution P (e.g., 1920×1080 pixels in our experiments), preserving the original aspect ratio. This allows the model to learn the iterative grounding task without a heavy computational burden of processing large images. Although this may increase the grounding difficulty when searching for small UI elements, it effectively trains the model to approximate the target’s location.

During inference, we employ a cursor-centric focusing strategy (CCF) for any image larger than the training resolution. CCF begins with a single step to get an initial coarse prediction on the full image. Then, it crops a P -size area centred on this initial prediction, thereby positioning the cursor at the centre of this focused view; in the following steps, the model conducts fine-grained movement within the cropped area. Here, P is the maximum resolution during training, and the following moving steps will not include the original large image in the interaction history.

This combined strategy enables GUI-CURSOR to learn a general interaction policy without a heavy computational burden, while performing accurate inference on higher-resolution displays.

3. Experiments

3.1. Experiment Settings

Implementation Details We implement GUI-CURSOR using two base models: Qwen2.5-VL-7B (Bai et al., 2025) and UI-TARS-1.5-7B (Qin et al., 2025), and we train both models using the same settings. We train GUI-CURSOR with a maximum of 250 steps. The learning rate is 10^{-6} , the batch size is 32, and 12 sample moving trajectories for each instruction. We set the maximum moving steps to 4 during training. Following Yang et al. (2025a), we use GUI grounding datasets from Aria-UI (Yang et al., 2025b) and OS-Atlas (Wu et al., 2024), employ the same filtering and preprocessing scripts, and randomly sample data to train the model. We apply online filtering (Cui et al., 2025) to filter out training examples when all sampled trajectories either successfully or unsuccessfully locate the target — a common data selection strategy to remove overly easy or difficult examples for online policy models. More implementation details are available in Appendix A.

Evaluation Benchmarks and Baseline Models We evaluate our method on the four GUI grounding benchmarks: ScreenSpot-V2 (Cheng et al., 2024; Wu et al., 2024), ScreenSpot-Pro (Li et al., 2025b), OSWorld-G (Xie et al., 2025), and UI-Vision (Nayak et al., 2025); we conduct online agentic evaluation using OSWorld (Xie et al., 2024). Moreover, we design a cursor-in-box test and use SpatialMQA (Liu et al., 2025a) and SPHERE (Zhang et al., 2025e) to evaluate the spatial reasoning capability. We compare GUI-CURSOR against GUI grounding models optimised by supervised fine-tuning: SeeClick (Cheng et al., 2024), UGround (Gou et al., 2025), OS-Atlas (Wu et al., 2024), UI-TARS (Qin et al., 2025) and GUI-Actor (Wu et al., 2025b), and RL methods: UI-R1 (Lu et al., 2025), GUI-R1 (Luo et al., 2025a), InfiGUI-R1 (Liu et al., 2025c), SE-GUI (Yuan et al., 2025), LPO (Tang et al., 2025b), GTA1 (Yang et al., 2025a), and GUI-G² (Tang et al., 2025a).

3.2. Main Experimental Results

Grounding Evaluation Table 1, Table 2, and Table 3 show the evaluation results on ScreenSpot-Pro, OSWorld-G, UI-Vision, and ScreenSpot-v2. GUI-CURSOR, based on UI-TARS-1.5-7B, obtains the best results in 3 out of 4 of the grounding benchmarks compared to other methods, except for OSWorld-G, with a small gap of 2 compared to GTA1; however, we show that GUI-CURSOR outperforms GTA1 on OSWorld in the later analysis. In the more challenging

Model	CAD		Dev		Creative		Scientific		Office		OS		Avg.		
	Text	Icon	Text	Icon	Text	Icon	Text	Icon	Text	Icon	Text	Icon	Text	Icon	Avg.
<i>Supervised Fine-Tuning Methods</i>															
SeeClick (Cheng et al., 2024)	0.6	0.0	1.0	0.0	2.5	0.0	3.5	0.0	1.1	0.0	2.8	0.0	1.8	0.0	1.1
OS-Atlas-7B (Wu et al., 2024)	33.1	1.4	28.8	2.8	12.2	4.7	37.5	7.3	33.9	5.7	27.1	4.5	28.1	4.0	18.9
UGround-7B (Gou et al., 2025)	-	-	-	-	-	-	-	-	-	-	-	-	-	-	31.1
UI-TARS-7B (Qin et al., 2025)	58.4	12.4	50.0	9.1	20.8	9.4	63.9	31.8	63.3	20.8	30.8	16.9	47.8	16.2	35.7
UI-TARS-72B (Qin et al., 2025)	63.0	17.3	57.1	15.4	18.8	12.5	64.6	20.9	63.3	26.4	42.1	15.7	50.9	17.5	38.1
Jedi-7B (Xie et al., 2025)	42.9	11.0	50.0	11.9	38.0	14.1	72.9	25.5	75.1	47.2	33.6	16.9	52.6	18.2	39.5
GUI-Actor-7B (Wu et al., 2025b)	-	-	-	-	-	-	-	-	-	-	-	-	-	-	47.7
<i>Reinforcement Learning Methods</i>															
UI-R1-3B (Lu et al., 2025)	11.2	6.3	22.7	4.1	27.3	3.5	42.4	11.8	32.2	11.3	13.1	4.5	24.9	6.4	17.8
UI-R1-E-3B (Lu et al., 2025)	37.1	12.5	46.1	6.9	41.9	4.2	56.9	21.8	65.0	26.4	32.7	10.1	-	-	33.5
GUI-R1-3B (Luo et al., 2025a)	26.4	7.8	33.8	4.8	40.9	5.6	61.8	17.3	53.6	17.0	28.1	5.6	-	-	-
InfGUI-R1-3B (Liu et al., 2025c)	33.0	14.1	51.3	12.4	44.9	7.0	58.3	20.0	65.5	28.3	43.9	12.4	49.1	14.1	35.7
GUI-R1-7B (Luo et al., 2025a)	23.9	6.3	49.4	4.8	38.9	8.4	55.6	11.8	58.7	26.4	42.1	16.9	-	-	-
SE-GUI-7B (Yuan et al., 2025)	51.3	42.2	68.2	19.3	57.6	9.1	75.0	28.2	78.5	43.4	49.5	25.8	63.5	21.0	47.3
GUI-G ² -7B (Tang et al., 2025a)	55.8	12.5	68.8	17.2	57.1	15.4	77.1	24.5	74.0	32.7	57.9	21.3	64.7	19.6	47.5
GTA1-7B (Yang et al., 2025a)	66.9	20.7	62.6	18.2	53.3	17.2	76.4	31.8	82.5	50.9	48.6	25.9	65.5	25.2	50.1
GUI-Cursor-7B (Qwen2.5-VL-7B)	80.5	33.1	65.7	18.2	62.4	25.0	83.3	32.7	84.2	43.4	65.4	31.5	73.3	29.3	56.5
GUI-Cursor-7B (UI-TARS-1.5-7B)	55.8	53.1	60.4	60.0	52.0	49.0	71.5	42.7	80.8	67.9	55.1	37.1	62.5	50.8	58.1

Table 1. ScreenSpot-Pro accuracy (%) broken down by six application domains (CAD, Dev, Creative, Scientific, Office, OS) and by text versus icon queries. GUI-CURSOR attains the best average and delivers the strongest icon grounding, notably on CAD, Creative, and OS screens, outperforming both supervised and reinforcement learning baselines.

Model	OSWorld-G							UI-Vision			
	Text Match	Element Rec.	Layout Und.	Fine-grained Manipulation	Refusal	Avg.		Basic	Func.	Spatial	Avg
OS-Atlas-7B (Wu et al., 2024)	44.1	29.4	35.2	16.8	7.4	27.7		12.2	11.2	3.7	9.0
UGround-v1-7B (Gou et al., 2025)	51.3	40.3	43.5	24.8	0	36.4		15.4	17.1	6.3	12.9
Jedi-7B (Xie et al., 2025)	65.9	55.5	57.7	46.9	7.4	54.1		-	-	-	-
Qwen2.5-VL-7B (Bai et al., 2025)	45.6	32.7	41.9	18.1	0	31.4		1.2	0.8	0.5	0.9
UI-TARS-1.5-7B (Qin et al., 2025)	67.3	64.5	65.2	42.9	0	61.9		22.9	26.1	6.6	18.1
<i>Initialised from Qwen2.5-VL-7B</i>											
GUI-Spotlight (Lei et al., 2025)	47.3	50.0	40.1	20.2	0	35.6		11.1	13.4	1.2	8.3
GUI-Cursor (Qwen2.5-VL-7B)	70.1	58.5	62.1	52.3	0	58.0		35.0	31.2	12.1	25.7
<i>Initialised from UI-TARS-1.5-7B</i>											
GTA1-7B (Yang et al., 2025a)	63.2	82.1	74.2	42.9	0	67.7		35.4	33.1	11.4	26.2
GUI-Spotlight (Lei et al., 2025)	68.2	60.6	63.2	45.6	0	62.7		32.1	30.2	9.1	23.4
GUI-Cursor (UI-TARS-1.5-7B)	77.0	66.7	70.4	67.4	0	65.6		35.4	33.5	14.2	27.3

Table 2. Evaluation results on OSWorld-G and UI-Vision. We find that GUI-Cursor achieves significantly higher accuracy on tasks that require understanding spatial relationships ("Spatial" and "Fine-grained Manipulation" categories).

benchmark ScreenSpot-Pro, which contains GUI images with high resolution and complex layouts, GUI-CURSOR outperforms the previous leading model GTA1 (Yang et al., 2025a) by 8.1%. GUI-CURSOR uses the same training setting as GTA1, so these results demonstrate the effectiveness of learning GUI grounding by moving a cursor. Moreover, we find GUI-CURSOR achieves significantly better accuracy on the tasks that require understanding and inferring spatial relationships: the "Spatial" category in UI-Vision and the "Manipulation" category in OSWorld-G. This indicates that GUI-CURSOR obtains better spatial reasoning capability by learning with spatial reasoning from visual feedback. A qualitative case analysis is provided in Appendix M.

Online Agentic Evaluation We conduct the online evaluation using OSWorld (Xie et al., 2024). In the agentic setting,

a planner model is applied to predict actions and call the grounding model to obtain the precise positions to take these actions. To compare the grounding models fairly, we use the same o3-based planning strategy (OpenAI, 2025) used by GTA1-7B (Yang et al., 2025a). The results shown in the Table 4 indicate that GUI-Cursor significantly improves agent performance. Using fewer action steps, GUI-Cursor-7B (57.1% at 50 steps) outperforms GTA1-7B (53.1% at 100 steps), and also outperforms the larger model GTA1-32B (55.4% at 100 steps). These results demonstrate that GUI-CURSOR can handle more realistic and diverse grounding tasks in agentic scenarios.

Ablation Studies *Trajectory penalties and thinking process improve the grounding accuracy.* In Figure 2, we show the accuracy of our model without using each of the penalty

Model	Mobile	Desktop	Web	Average
<i>Supervised Fine-Tuning Methods</i>				
SeeClick (Cheng et al., 2024)	64.6	49.7	43.9	55.1
OmniParser-v2 (Lu et al., 2024)	85.1	76.6	73.8	80.7
OS-Atlas-7B (Wu et al., 2024)	85.5	77.2	84.0	84.1
UGround (Gou et al., 2025)	89.2	86.4	84.7	87.6
UI-TARS-7B (Qin et al., 2025)	93.0	90.2	89.4	91.6
UI-TARS-72B (Qin et al., 2025)	90.6	89.6	89.6	90.3
Jedi-7B (Xie et al., 2025)	92.1	91.9	89.3	91.7
GUI-Actor-7B (Wu et al., 2025b)	93.3	91.9	90.2	92.5
<i>Reinforcement Learning Methods</i>				
LPO-8B (Tang et al., 2025b)	90.4	91.2	89.9	90.5
SE-GUI-7B (Yuan et al., 2025)	-	-	-	90.3
GUI-G ² -7B (Tang et al., 2025a)	95.1	92.4	90.9	93.3
GTA1-7B (Yang et al., 2025a)	93.8	92.1	89.5	92.4
GUI-Cursor-7B (Qwen2.5-VL-7B)	94.9	92.9	92.6	93.9
GUI-Cursor-7B (UI-TARS-1.5-7B)	93.3	95.8	91.7	93.9

Table 3. Accuracy on ScreenSpot-v2.

Agent Model	Accuracy		
	15 steps	50 steps	100 steps
o3 (OpenAI, 2025)	9.1	17.2	23.0
GTA1-7B w/ o3 (Yang et al., 2025a)	-	-	53.1
GTA1-32B w/ o3 (Yang et al., 2025a)	-	-	55.4
GUI-Cursor-7B (Qwen2.5-VL-7B) w/ o3	-	57.1	-
GUI-Cursor-7B (UI-TARS-1.5-7B) w/ o3	-	55.2	-

Table 4. Online evaluation results on OSWorld.

terms, and without generating thinking tokens. The results show that the accuracy without each penalty is lower than the full reward, and it is less effective in improving accuracy with additional movement steps. Additionally, we find that without thinking, the accuracy of GUI-CURSOR decreases significantly, whereas Tang et al. (2025a) and Yang et al. (2025a) found that thinking is less effective in improving accuracy. One possible reason might be that GUI-CURSOR is trained in an interactive environment – the explicit analysis of the target and the spatial relationship between the target and the cursor, which is helpful for the model to check whether the cursor is correctly located at the target, can help to align the coordinates with their corresponding position on the GUI. The above results show that both reward penalties (Section 2.3) and the generation of explicit spatial reasoning are essential for the downstream accuracy of the model.

Cursor-centric focusing improves the grounding accuracy. Figure 3 shows the accuracy without using cursor-centric focusing (w/o CCF). We find CCF in ScreenSpot-Pro is more effective (45.3% $\rightarrow$ 56.5%) than ScreenSpot-v2 (93.6% $\rightarrow$ 93.9%), demonstrating the effectiveness of CCF on large and complex GUI images. We also present the two-stage inference strategy used in GUI-Actor (Wu et al., 2025b), where the model first predicts a set of candidate positions and an external verifier model is applied to select the final answer. Though the verifier introduces new parameters and needs additional training, the accuracy improvement (44.6% $\rightarrow$ 47.7%, red bars) is less effective than CCF. To further verify the effectiveness of CCF, we apply a similar

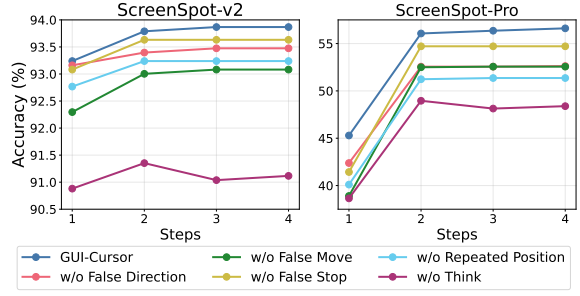


Figure 2. Ablations on reward penalties and thinking

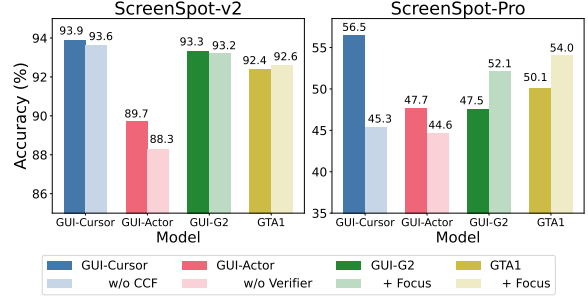


Figure 3. Ablations and comparison on CCF

approach to the SOTA methods GTA1 (Yang et al., 2025a) and GUI-G² (Tang et al., 2025a), labelled with (+ Focus) in Figure 3. Here, the image is cropped around the initial prediction using the same image size as in GUI-CURSOR, and the model makes its second step prediction using this cropped image. Results show that the accuracy of GTA1 is effectively improved (50.1% $\rightarrow$ 54.0%, yellow bars), but it is still lower than that of GUI-CURSOR (56.5%). However, without CCF, the accuracy of GUI-CURSOR is lower than GTA1, which is because GTA1 is trained with a higher resolution setting (4096×2160 pixels) than GUI-CURSOR (1920×1080 pixels). The above results show that CCF is an effective method to balance the training efficiency and inference accuracy.

Analysis on the Cursor Movements GUI-CURSOR *adaptively conducts more cursor movement on more difficult tasks*. As shown in Figure 4, GUI-CURSOR conducts single-step interactions on most instances after CCF: it executes more than one step movement on 0.5% of examples in ScreenSpot-v2, but this rate increases to 9.4% on the more difficult dataset ScreenSpot-Pro. In Figure 5, we present the average size of the target in the samples where the model conducts multi-step (red bar) and one-step movement (blue bar). On average, the target size for multi-step samples is 5024 pixels, compared to 31584 pixels for one-step samples. This result indicates that the model may conduct more steps when the target is small. More details and additional analyses are available in Appendix C.

Trajectory penalties influence the number of movement steps

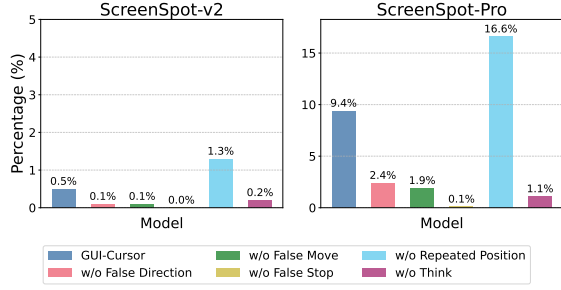


Figure 4. Percentage of examples with > 1 steps movement.

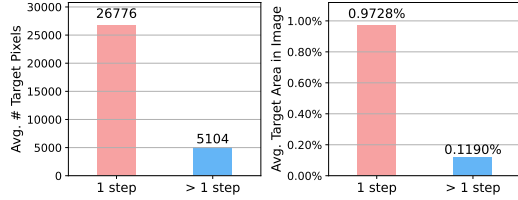


Figure 5. The target size in one- and multi-step movement cases.

and training dynamics. Compared to the model without the repeated position penalty, the percentage of cursor moving steps increases: 0.5% $\rightarrow$ 1.3% in ScreenSpot-v2 and 9.4% $\rightarrow$ 16.6% in ScreenSpot-Pro, indicating that the repeated position penalty can improve efficiency. The model without the false stop penalty tends not to perform multiple cursor movements: 0.5% $\rightarrow$ 0.0% in ScreenSpot-v2, and 9.4% $\rightarrow$ 0.1% in ScreenSpot-Pro, indicating that the false stop penalty prevents a degenerate behaviour where the model always takes a single step. Figure 8 presents the average number of movement steps and the overall reward in each batch during training. GUI-CURSOR initially converges to single-step predictions after 50 training steps; at later steps, the average number of movements increases to an average of 1.25. The model without the false stop penalty quickly converges to take one-step movements around 20 steps and fail to learn to move at later steps, which further shows the false stop penalty is necessary for the model to learn multi-step movements. The model without the repeated position penalty converges to a single-step policy more slowly than GUI-CURSOR, and it becomes unstable when trained for more steps, with the number of steps increasing significantly around 220 steps. A more detailed analysis of training dynamics is available in Appendix D.

3.3. Analysis and Discussions on Spatial Reasoning

Moving the Cursor for GUI Grounding without Fine-Tuning We investigate whether general-purpose VLMs can conduct GUI grounding by moving a cursor without fine-tuning. This zero-shot task assesses the model’s spatial reasoning ability to identify the spatial relationship between a cursor and a target and refine its predictions accord-

ingly. We test two distinct prompting strategies: 1) *Relative Move*: the model is prompted to generate a relative offset $(\Delta x, \Delta y)$, and the cursor is then moved from its current position (x_t, y_t) to $(x_t + \Delta x, y_t + \Delta y)$; and 2) *Direct Move*: the model is prompted to generate new absolute coordinates (x, y) , and the cursor is rendered at that position — this is the strategy used by GUI-CURSOR in the main experiments (Section 3.2). More implementation details and analysis are presented in Appendix B.

We analyse the accuracy for GPT-4o (OpenAI, 2024) and Qwen2.5-VL-7B on ScreenSpot-v2. In the standard one-step GUI grounding setting, the accuracy of GPT-4o is low (17.5%), and Qwen2.5-VL-7B performs well (88.8%) because it has been fine-tuned on GUI grounding tasks. With 10 steps of movement, GPT-4o’s performance improves with both direct move and (17.5% $\rightarrow$ 21.7%) relative move (17.5% $\rightarrow$ 25.5%), suggesting it possesses the underlying spatial reasoning capability to benefit from the iterative process. However, the same strategies cause a significant performance drop for the Qwen2.5-VL-7B (direct move: 36.3%, relative move: 1.3%). The above analysis suggests that *the high single-step accuracy of Qwen2.5-VL-7B does not generalise to this iterative process, and its success on GUI grounding may not be founded on robust spatial understanding of the GUI image*. Based on this result, it is necessary to fine-tune the model to gain the ability to use a cursor. We use the direct move strategy in GUI-CURSOR, as its higher zero-shot accuracy provides a better starting point for reinforcement learning.

Probing Spatial Reasoning We designed a *cursor-in-box* test to isolate and evaluate a VLM’s spatial reasoning accuracy over different positions of the image. The task is straightforward: given a white background containing a red bounding box and a black cursor, the model must answer “Yes or No” to the question: “Is the black cursor inside the red bounding box?”. We generated a comprehensive dataset by placing the box at different locations across the image and sampling cursor positions both inside and outside the box for each location. More implementation details and analysis are presented in Appendix E. This minimalist setup eliminates errors that stem from target misidentification in standard GUI grounding by moving a cursor, allowing us to focus on evaluating spatial reasoning around the cursor.

We evaluate Qwen-2.5-VL-7B and measure performance using the F1 score, visualised in Figure 6. The results reveal a critical flaw in Qwen-2.5-VL-7B. The left heatmap shows that the model struggles with this simple task and also exhibits a severe positional bias. It performs well only when the box is near the centre of the image (green areas) and fails towards the edges (red areas). This failure suggests its grounding capabilities may be brittle and not founded on a robust spatial understanding of GUI images. The right

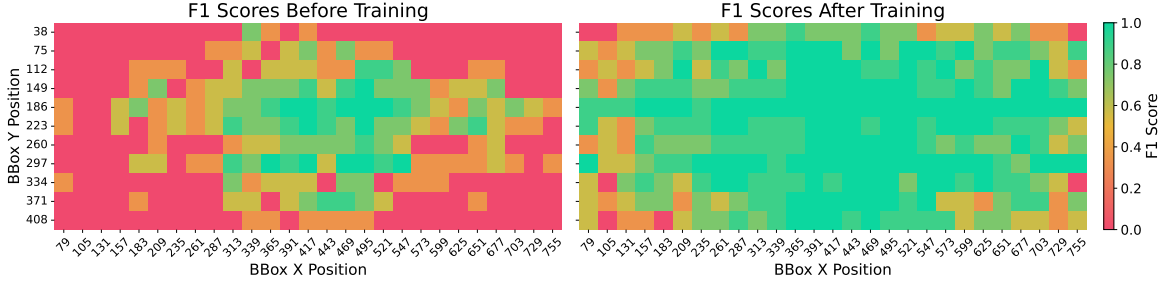


Figure 6. Cursor-in-box spatial reasoning test results. The heatmap presents F1 scores across different positions. Left: before training, Qwen-2.5-VL-7B exhibits a strong centre bias; Right: GUI-CURSOR achieves higher F1, despite not being explicitly trained on this task.

Model	SPHERE				SpatialMQA
	Single Skill	Reason.	Combine 2 Skills	Avg.	
Qwen2.5-VL-7B (Bai et al., 2025)	70.9	54.7	39.0	56.7	38.1
GUI-G ² -7B (Tang et al., 2025a)	71.2	55.8	39.9	57.3	38.0
GUI-Cursor (Qwen2.5-VL-7B)	71.2	56.6	42.9	58.5	43.4

Table 5. Evaluation results on spatial reasoning benchmarks SPHERE (Zhang et al., 2025e) and SpatialMQA (Liu et al., 2025a).

heatmap shows that GUI-CURSOR achieves higher accuracy, though it is not explicitly trained on this classification task. Instead, we design trajectory penalties to discourage the wrong-moving behaviours, which may implicitly improve this spatial reasoning capability by rewarding the trajectory with a correct spatial thinking process.

Generalisation of Spatial Reasoning We further evaluate the model’s spatial reasoning capability on SpatialMQA (Liu et al., 2025a) and SPHERE (Zhang et al., 2025e). SpatialMQA evaluates the accuracy of inferring the spatial relationship between two objects. Though it evaluates spatial reasoning on 3D natural images, we find that GUI-CURSOR shows improvement in such out-of-distribution settings (+5.3%). While GUI-G²-7B (Tang et al., 2025a), fine-tuned from the same model as ours using one-step RL, does not show this improvement. SPHERE evaluates the model’s spatial understanding across multiple aspects, and we find our method yields more improvements in the “reasoning” (+1.9%) and “combine 2 skills” (+3.9%) categories. The above results provide additional evidence that GUI-CURSOR obtains better spatial reasoning capability and the ability to utilise multiple spatial reasoning skills in out-of-distribution domains beyond the GUI images.

4. Related Work

GUI Grounding A common implementation for these agents involves a planning model that determines the sequence of actions, and a grounding model that predicts the positions to execute them. Recently, the purely vision-driven approach has shown significant advantages in accuracy and

general applicability, as it does not rely on backends of systems and external utilities (Gou et al., 2025; Wu et al., 2025b; Yang et al., 2025a). Some works train grounding models by SFT on large-scale GUI datasets (Hong et al., 2024; Gou et al., 2025; Wu et al., 2024; Xu et al., 2024), and some works train the models by RL with rule-based rewards (Luo et al., 2025a; Zhou et al., 2025b; Lu et al., 2025; Liu et al., 2025c). Some concurrent works learn iterative cropping, aiming to reduce the difficulty of one-step grounding on large images (Lei et al., 2025; Ye et al., 2025; Du et al., 2025). Differently, GUI-CURSOR aims to improve the alignment between coordinates and on-screen locations by learning to control a cursor; the cropping CCF is only applied during inference and can also improve the accuracy of other methods due to its simplicity and effectiveness.

Multimodal Reasoning Huang et al. (2025); Zhou et al. (2025a); Shen et al. (2025a); Chen et al. (2025a); Su et al. (2025a) show emergent multimodal reasoning capability in VLMs after RL training. Chen et al. (2025b); Liu et al. (2025b); Zhang et al. (2025d) reveal that VLMs struggle with spatial reasoning and often fail in inferring spatial relationships between objects. (Wu et al., 2025a) improves image-text interleaved reasoning through RL. Li et al. (2025a) and Chern et al. (2025) proposed to generate visual reasoning thoughts. Our multi-step interaction learning process is related to recent works on multi-turn interaction paradigms in VLMs (Shen et al., 2025b; Su et al., 2025b), which suggested scaling test-time interaction to improve reasoning, and tool-augmented reasoning frameworks (Yao et al., 2023; Qin et al., 2024; Qu et al., 2025), where agents iteratively refine their predictions by sequential actions.

5. Conclusion

We reframe GUI grounding as a cursor-driven search. During training, the rendered cursor provides explicit visual feedback that shows the predicted position on the GUI image, helping the model better align numerical coordinates with their corresponding on-screen positions. During inference, this interactive environment allows the model to

incrementally refine its predictions. GUI-CURSOR couples GRPO with a trajectory-aware reward and employs cursor-centric focusing (CCF) for balancing training efficiency and inference accuracy. Our comprehensive experiments demonstrate the effectiveness of GUI-CURSOR. Moreover, the evaluation results on spatial reasoning tasks show that GUI-CURSOR obtains better spatial reasoning capability that also generalises to out-of-distribution images.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgements

We would like to thank the anonymous reviewers for their thorough feedback and suggestions. The work was done during Yu Zhao’s internship at Microsoft Research Cambridge, UK. Yu Zhao was partly supported by the UKRI Centre for Doctoral Training in Natural Language Processing, funded by UK Research and Innovation (grant EP/S022481/1) and the University of Edinburgh, School of Informatics.

References

- Bai, S., Chen, K., Liu, X., Wang, J., Ge, W., Song, S., Dang, K., Wang, P., Wang, S., Tang, J., Zhong, H., Zhu, Y., Yang, M., Li, Z., Wan, J., Wang, P., Ding, W., Fu, Z., Xu, Y., Ye, J., Zhang, X., Xie, T., Cheng, Z., Zhang, H., Yang, Z., Xu, H., and Lin, J. Qwen2.5-vl technical report. *CoRR*, abs/2502.13923, 2025. doi: 10.48550/ARXIV.2502.13923. URL <https://doi.org/10.48550/arXiv.2502.13923>.
- Chen, H., Tu, H., Wang, F., Liu, H., Tang, X., Du, X., Zhou, Y., and Xie, C. Sft or rl? an early investigation into training rl-like reasoning large vision-language models. *ArXiv*, abs/2504.11468, 2025a. URL <https://api.semanticscholar.org/CorpusID:277824294>.
- Chen, S., Zhu, T., Zhou, R., Zhang, J., Gao, S., Niebles, J. C., Geva, M., He, J., Wu, J., and Li, M. Why is spatial reasoning hard for vlms? an attention mechanism perspective on focus areas. *ArXiv*, abs/2503.01773, 2025b. URL <https://api.semanticscholar.org/CorpusID:276775433>.
- Cheng, K., Sun, Q., Chu, Y., Xu, F., Li, Y., Zhang, J., and Wu, Z. SeeClick: Harnessing GUI grounding for advanced visual GUI agents. In Ku, L., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, pp. 9313–9332. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.505. URL <https://doi.org/10.18653/v1/2024.acl-long.505>.
- Chern, E., Hu, Z., Chern, S., Kou, S., Su, J., Ma, Y., Deng, Z., and Liu, P. Thinking with generated images. *ArXiv*, abs/2505.22525, 2025. URL <https://api.semanticscholar.org/CorpusID:278959478>.
- Cui, G., Yuan, L., Wang, Z., Wang, H., Li, W., He, B., Fan, Y., Yu, T., Xu, Q., Chen, W., Yuan, J., Chen, H., Zhang, K., Lv, X., Wang, S., Yao, Y., Han, X., Peng, H., Cheng, Y., Liu, Z., Sun, M., Zhou, B., and Ding, N. Process reinforcement through implicit rewards. *CoRR*, abs/2502.01456, 2025.
- DeepSeek-AI. Deepseek-rl: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, abs/2501.12948, 2025. doi: 10.48550/ARXIV.2501.12948. URL <https://doi.org/10.48550/arXiv.2501.12948>.
- Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., and Su, Y. Mind2web: Towards a generalist agent for the web. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.
- Du, Y., Yan, Y., Tang, F., Lu, Z., Zong, C., Lu, W., Jiang, S., and Shen, Y. Test-time reinforcement learning for gui grounding via region consistency. *arXiv preprint arXiv:2508.05615*, 2025.
- Gou, B., Wang, R., Zheng, B., Xie, Y., Chang, C., Shu, Y., Sun, H., and Su, Y. Navigating the digital world as humans do: Universal visual grounding for GUI agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=kxnoqaisCT>.
- Hong, W., Wang, W., Lv, Q., Xu, J., Yu, W., Ji, J., Wang, Y., Wang, Z., Dong, Y., Ding, M., and Tang, J. Cogagent: A visual language model for GUI agents. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*, pp. 14281–14290. IEEE, 2024. doi: 10.1109/CVPR52733.2024.01354. URL <https://doi.org/10.1109/CVPR52733.2024.01354>.

- Huang, W., Jia, B., Zhai, Z., Cao, S., Ye, Z., Zhao, F., Xu, Z., Hu, Y., and Lin, S. Vision-rl: Incentivizing reasoning capability in multimodal large language models. *ArXiv*, abs/2503.06749, 2025. URL <https://api.semanticscholar.org/CorpusID:276902576>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In Flinn, J., Seltzer, M. I., Druschel, P., Kaufmann, A., and Mace, J. (eds.), *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pp. 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Lei, B., Xu, N., Payani, A., Hong, M., Liao, C., Cao, Y., and Ding, C. \textsc {GUI-Spotlight}: Adaptive iterative focus refinement for enhanced gui visual grounding. *arXiv preprint arXiv:2510.04039*, 2025.
- Li, C., Wu, W., Zhang, H., Xia, Y., Mao, S., Dong, L., Vulic, I., and Wei, F. Imagine while reasoning in space: Multimodal visualization-of-thought. *ArXiv*, abs/2501.07542, 2025a. URL <https://api.semanticscholar.org/CorpusID:275471612>.
- Li, K., Meng, Z., Lin, H., Luo, Z., Tian, Y., Ma, J., Huang, Z., and Chua, T. Screenspot-pro: GUI grounding for professional high-resolution computer use. *CoRR*, abs/2504.07981, 2025b. doi: 10.48550/ARXIV.2504.07981. URL <https://doi.org/10.48550/arXiv.2504.07981>.
- Li, W., Bishop, W. E., Li, A., Rawles, C., Campbell-Ajala, F., Tyamagundlu, D., and Riva, O. On the effects of data scale on UI control agents. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.
- Liao, Z., Lu, Y., Gou, B., Sun, H., and Awadallah, A. Beyond clicking: a step towards generalist GUI grounding via text dragging. *CoRR*, abs/2601.06031, 2026. doi: 10.48550/ARXIV.2601.06031. URL <https://doi.org/10.48550/arXiv.2601.06031>.
- Liu, J., Liu, Z., Cen, Z., Zhou, Y., Zou, Y., Zhang, W., Jiang, H., and Ruan, T. Can multimodal large language models understand spatial relations? In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2025, Vienna, Austria, July 27 - August 1, 2025, pp. 620–632. Association for Computational Linguistics, 2025a. URL <https://aclanthology.org/2025.acl-long.31/>.
- Liu, J., Liu, Z., Cen, Z., Zhou, Y., Zou, Y., Zhang, W., Jiang, H., and Ruan, T. Can multimodal large language models understand spatial relations? *ArXiv*, abs/2505.19015, 2025b. URL <https://api.semanticscholar.org/CorpusID:278905162>.
- Liu, Y., Li, P., Xie, C., Hu, X., Han, X., Zhang, S., Yang, H., and Wu, F. Infigui-rl: Advancing multimodal GUI agents from reactive actors to deliberative reasoners. *CoRR*, abs/2504.14239, 2025c. doi: 10.48550/ARXIV.2504.14239. URL <https://doi.org/10.48550/arXiv.2504.14239>.
- Lu, Y., Yang, J., Shen, Y., and Awadallah, A. Omniparser for pure vision based GUI agent. *CoRR*, abs/2408.00203, 2024. doi: 10.48550/ARXIV.2408.00203. URL <https://doi.org/10.48550/arXiv.2408.00203>.
- Lu, Z., Chai, Y., Guo, Y., Yin, X., Liu, L., Wang, H., Xiong, G., and Li, H. UI-R1: enhancing action prediction of GUI agents by reinforcement learning. *CoRR*, abs/2503.21620, 2025. doi: 10.48550/ARXIV.2503.21620. URL <https://doi.org/10.48550/arXiv.2503.21620>.
- Luo, R., Wang, L., He, W., and Xia, X. GUI-R1 : A generalist rl-style vision-language action model for GUI agents. *CoRR*, abs/2504.10458, 2025a. doi: 10.48550/ARXIV.2504.10458. URL <https://doi.org/10.48550/arXiv.2504.10458>.
- Luo, T., Logeswaran, L., Johnson, J., and Lee, H. Visual test-time scaling for GUI agent grounding. In *IEEE/CVF International Conference on Computer Vision, ICCV 2025, Honolulu, HI, USA, October 19-25, 2025*, pp. 19989–19998. IEEE, 2025b. doi: 10.1109/ICCV51701.2025.01859. URL <https://doi.org/10.1109/ICCV51701.2025.01859>.
- Nayak, S., Jian, X., Lin, K. Q., Rodriguez, J. A., Kalsi, M., Chapados, N., Özsu, M. T., Agrawal, A., Vazquez, D., Pal, C., Taslakian, P., Gella, S., and Rajeswar, S. UI-vision: A desktop-centric GUI benchmark for visual perception and interaction. In Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K., and Zhu, J. (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 45817–45851. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/nayak25a.html>.
- OpenAI. Gpt-4o system card. *ArXiv*, abs/2410.21276, 2024. URL <https://api.semanticscholar.org/CorpusID:273662196>.

- OpenAI, T. Introducing openai o3 and o4-mini. <https://openai.com/index/introducing-o3-and-o4-mini/>, 2025.
- Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerstein, M., Li, D., Liu, Z., and Sun, M. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=dHng200Jjr>.
- Qin, Y., Ye, Y., Fang, J., Wang, H., Liang, S., Tian, S., Zhang, J., Li, J., Li, Y., Huang, S., Zhong, W., Li, K., Yang, J., Miao, Y., Lin, W., Liu, L., Jiang, X., Ma, Q., Li, J., Xiao, X., Cai, K., Li, C., Zheng, Y., Jin, C., Li, C., Zhou, X., Wang, M., Chen, H., Li, Z., Yang, H., Liu, H., Lin, F., Peng, T., Liu, X., and Shi, G. UI-TARS: pioneering automated GUI interaction with native agents. *CoRR*, abs/2501.12326, 2025. doi: 10.48550/ARXIV.2501.12326. URL <https://doi.org/10.48550/arXiv.2501.12326>.
- Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., and Wen, J. Tool learning with large language models: a survey. *Frontiers Comput. Sci.*, 19(8):198343, 2025. doi: 10.1007/S11704-024-40678-2. URL <https://doi.org/10.1007/s11704-024-40678-2>.
- Qwen Team. Qwen3-vl technical report. *CoRR*, abs/2511.21631, 2025. doi: 10.48550/ARXIV.2511.21631. URL <https://doi.org/10.48550/arXiv.2511.21631>.
- Shen, H., Liu, P., Li, J., Fang, C., Ma, Y., Liao, J., Shen, Q., Zhang, Z., Zhao, K., Zhang, Q., Xu, R., and Zhao, T. Vlm-r1: A stable and generalizable r1-style large vision-language model. *ArXiv*, abs/2504.07615, 2025a. URL <https://api.semanticscholar.org/CorpusID:277667819>.
- Shen, J., Bai, H., Zhang, L., Zhou, Y., Setlur, A. R., Tong, S., Caples, D., Jiang, N., Zhang, T., Talwalkar, A., and Kumar, A. Thinking vs. doing: Agents that reason by scaling test-time interaction. *ArXiv*, abs/2506.07976, 2025b. URL <https://api.semanticscholar.org/CorpusID:279251333>.
- Su, A., Wang, H., Ren, W., Lin, F., and Chen, W. Pixel reasoner: Incentivizing pixel-space reasoning with curiosity-driven reinforcement learning. *ArXiv*, abs/2505.15966, 2025a. URL <https://api.semanticscholar.org/CorpusID:278789415>.
- Su, H., Sun, R., Yoon, J., Yin, P., Yu, T., and Arik, S. Ö. Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments. *ArXiv*, abs/2501.10893, 2025b. URL <https://api.semanticscholar.org/CorpusID:275757520>.
- Tang, F., Gu, Z., Lu, Z., Liu, X., Shen, S., Meng, C., Wang, W., Zhang, W., Shen, Y., Lu, W., Xiao, J., and Zhuang, Y. Gui-g²: Gaussian reward modeling for gui grounding, 2025a. URL <https://arxiv.org/abs/2507.15846>.
- Tang, J., Xia, Y., Wu, Y., Hu, Y., Chen, Y., Chen, Q., Xu, X., Wu, X., Lu, H., Ma, Y., Lu, S., and Chen, Q. LPO: towards accurate GUI agent interaction via location preference optimization. *CoRR*, abs/2506.09373, 2025b. doi: 10.48550/ARXIV.2506.09373. URL <https://doi.org/10.48550/arXiv.2506.09373>.
- Wu, J., Guan, J., Feng, K., Liu, Q., Wu, S., Wang, L., Wu, W., and Tan, T. Reinforcing spatial reasoning in vision-language models with interwoven thinking and visual drawing. *ArXiv*, abs/2506.09965, 2025a. URL <https://api.semanticscholar.org/CorpusID:279306073>.
- Wu, Q., Cheng, K., Yang, R., Zhang, C., Yang, J., Jiang, H., Mu, J., Peng, B., Qiao, B., Tan, R., Qin, S., Liden, L., Lin, Q., Zhang, H., Zhang, T., Zhang, J., Zhang, D., and Gao, J. Gui-actor: Coordinate-free visual grounding for GUI agents. *CoRR*, abs/2506.03143, 2025b. doi: 10.48550/ARXIV.2506.03143. URL <https://doi.org/10.48550/arXiv.2506.03143>.
- Wu, Z., Wu, Z., Xu, F., Wang, Y., Sun, Q., Jia, C., Cheng, K., Ding, Z., Chen, L., Liang, P. P., and Qiao, Y. OS-ATLAS: A foundation action model for generalist GUI agents. *CoRR*, abs/2410.23218, 2024. doi: 10.48550/ARXIV.2410.23218. URL <https://doi.org/10.48550/arXiv.2410.23218>.
- Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., and Yu, T. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.
- Xie, T., Deng, J., Li, X., Yang, J., Wu, H., Chen, J., Hu, W., Wang, X., Xu, Y., Wang, Z., Xu, Y., Wang, J., Sahoo, D., Yu, T., and Xiong, C. Scaling computer-use grounding via user interface decomposition and synthesis.

- CoRR, abs/2505.13227, 2025. doi: 10.48550/ARXIV.2505.13227. URL <https://doi.org/10.48550/arXiv.2505.13227>.
- Xu, Y., Wang, Z., Wang, J., Lu, D., Xie, T., Saha, A., Sahoo, D., Yu, T., and Xiong, C. Aguviz: Unified pure vision agents for autonomous GUI interaction. CoRR, abs/2412.04454, 2024. doi: 10.48550/ARXIV.2412.04454. URL <https://doi.org/10.48550/arXiv.2412.04454>.
- Yang, Y., Li, D., Dai, Y., Yang, Y., Luo, Z., Zhao, Z., Hu, Z., Huang, J., Saha, A., Chen, Z., et al. Gta1: Gui test-time scaling agent. *arXiv preprint arXiv:2507.05791*, 2025a.
- Yang, Y., Wang, Y., Li, D., Luo, Z., Chen, B., Huang, C., and Li, J. Aria-ui: Visual grounding for GUI instructions. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pp. 22418–22433. Association for Computational Linguistics, 2025b. URL <https://aclanthology.org/2025.findings-acl.1152/>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Ye, X., Li, Y., Dai, W., Liu, M., Chen, Z., Han, Z., Min, H., Ren, J., Zhang, X., Yang, W., et al. Gui-arp: Enhancing grounding with adaptive region perception for gui agents. *arXiv preprint arXiv:2509.15532*, 2025.
- Yuan, X., Zhang, J., Li, K., Cai, Z., Yao, L., Chen, J., Wang, E., Hou, Q., Chen, J., Jiang, P., and Li, B. Enhancing visual grounding for GUI agents via self-evolutionary reinforcement learning. CoRR, abs/2505.12370, 2025. doi: 10.48550/ARXIV.2505.12370. URL <https://doi.org/10.48550/arXiv.2505.12370>.
- Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., and Zhang, Q. Large language model-brained GUI agents: A survey. *Trans. Mach. Learn. Res.*, 2025, 2025a. URL <https://openreview.net/forum?id=xChvYjvXTp>.
- Zhang, C., Huang, H., Ni, C., Mu, J., Qin, S., He, S., Wang, L., Yang, F., Zhao, P., Du, C., Li, L., Kang, Y., Jiang, Z., Zheng, S., Wang, R., Qian, J., Ma, M., Lou, J., Lin, Q., Rajmohan, S., and Zhang, D. UFO2: the desktop agentos. CoRR, abs/2504.14603, 2025b. doi: 10.48550/ARXIV.2504.14603. URL <https://doi.org/10.48550/arXiv.2504.14603>.
- Zhang, C., Li, L., He, S., Zhang, X., Qiao, B., Qin, S., Ma, M., Kang, Y., Lin, Q., Rajmohan, S., Zhang, D., and Zhang, Q. UFO: A ui-focused agent for windows OS interaction. In Chiruzzo, L., Ritter, A., and Wang, L. (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, pp. 597–622. Association for Computational Linguistics, 2025c. doi: 10.18653/V1/2025.NAACL-LONG.26. URL <https://doi.org/10.18653/v1/2025.naacl-long.26>.
- Zhang, W., Huang, Y., Xu, Y., Huang, J., Zhi, H., Ren, S., Xu, W., and Zhang, J. Why do mlms struggle with spatial understanding? a systematic analysis from data to architecture. *arXiv preprint arXiv:2509.02359*, 2025d.
- Zhang, W., Ng, W. E., Ma, L., Wang, Y., Zhao, J., Koenecke, A., Li, B., and Wang, L. SPHERE: unveiling spatial blind spots in vision-language models through hierarchical evaluation. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pp. 11591–11609. Association for Computational Linguistics, 2025e. URL <https://aclanthology.org/2025.acl-long.568/>.
- Zheng, Y., Lu, J., Wang, S., Feng, Z., Kuang, D., and Xiong, Y. Easyrl: An efficient, scalable, multi-modality rl training framework. <https://github.com/hiyouga/EasyRl>, 2025.
- Zhou, H., Li, X., Wang, R., Cheng, M., Zhou, T., and Hsieh, C.-J. R1-zero’s ”aha moment” in visual reasoning on a 2b non-sft model. *ArXiv*, abs/2503.05132, 2025a. URL <https://api.semanticscholar.org/CorpusID:276884980>.
- Zhou, Y., Dai, S., Wang, S., Zhou, K., Jia, Q., and Xu, J. GUI-G1: understanding r1-zero-like training for visual grounding in GUI agents. CoRR, abs/2505.15810, 2025b. doi: 10.48550/ARXIV.2505.15810. URL <https://doi.org/10.48550/arXiv.2505.15810>.

A. Implementation Details

A.1. GRPO Training Objective

We optimise the policy π_θ , i.e., a vision-language model, to locate the target UI element with multi-step interaction using GRPO. For each instruction I , we sample a batch of N trajectories $\{\tau_1, \dots, \tau_N\}$ from the current policy, where each trajectory τ_i is a sequence of generated responses at each step: $\tau_i = \{A_0^i, A_1^i, \dots, A_T^i\}$, and each response A_t^i consists of tokens with a thinking process and an answer, as introduced in Section 2.2. We elaborate the rollout process in Appendix A.2. We calculate the reward R for each sampled trajectory: $\{R_{\tau_1}, R_{\tau_2}, \dots, R_{\tau_N}\}$, where R_{τ_i} refers to the overall reward for the trajectory τ_i . The reward R_{τ_i} is calculated by the weighted sum of the trajectory reward and a format reward $R_\tau = 0.9 \times R_T + 0.1 \times R_{\text{format}}$, where R_{format} is 1 when the output format follows the thinking before answering format, otherwise it is 0. Then, we calculate the relative advantage $\hat{A}_{\tau_i}$ of each sampled trajectory:

$$\hat{A}_{\tau_i} = \frac{R_{\tau_i} - \text{Mean}(\{R_{\tau_1}, R_{\tau_2}, \dots, R_{\tau_N}\})}{\text{Std}(\{R_{\tau_1}, R_{\tau_2}, \dots, R_{\tau_N}\})}. \quad (7)$$

The objective function seeks to increase the likelihood of high-reward trajectories:

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_i \min \left(r_i(\theta) \hat{A}_{\tau_i}, \text{clip}(r_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{\tau_i} \right) \right], \quad (8)$$

where, $r_i(\theta) = \frac{\prod_{t=1}^T \pi_\theta(A_t^i | I, O_0, A_0, \dots, O_{t-1})}{\prod_{t=1}^T \pi_{\theta_{\text{old}}}(A_t^i | I, O_0, A_0, \dots, O_{t-1})}$ is the probability ratio for responses in the trajectory τ_i , ϵ controls the trust region. We omit the KL regularisation term as previous work shows that it does not improve the grounding accuracy (Yang et al., 2025a).

A.2. Moving Trajectory Rollout

Given an instruction I and a GUI screenshot O , we rollout N different trajectories $\{\tau_i\}_{i=1}^N$ for each GRPO optimisation step by the following process: At the first step, we sample N different responses $\{A_0^i\}_{i=1}^N$ conditioned on the same initial history (I, O_0) , where each response has a thinking process before the action, and we do not constrain the action to be different. At the subsequent steps, we sample 1 response A_t^i conditioned on the interaction history $(I, O_0, A_0^i, \dots, A_{t-1}^i, O_t^i)$. Each trajectory τ_i stops growing until the action part in A_t^i is STOP, or the maximum number of steps T is reached. We use vLLM (Kwon et al., 2023) for efficient inference during rollout.

A.3. Hyperparameter Settings

GUI-CURSOR moves a cursor to locate the target. We use a cursor image with a size of 20×31 , and we present it in Figure 12. We set the weight of the trajectory penalty w_p to 0.2 (Equation (6)) in our experiments. We empirically observe that the false stop penalty is crucial for preventing the model from only conducting a single prediction without using the visual feedback, which may be because it results in a higher reward for the trajectories with accurate movements. We find that the accuracy improves slightly when increasing the weight of the false stop penalty r_{FS} to 0.5 and keeping other penalties 0.2. We train the model using 8 NVIDIA A100 80GB GPUs. We implement RL training based on EasyR1 (Zheng et al., 2025). During training, we evaluate the model on the validation set every 20 steps and save a checkpoint every 50 steps. We measure the success rate on the validation set, where a sample is considered successful if the final predicted position is within the target area and the final action is STOP. In our main experiments, we use the model at the 200th step, as the success rate does not improve in later steps, as shown in the third figure of Figure 10. Because sampling multi-step moving trajectories is time-consuming, we set the maximum number of movement steps to 4 during training. We find that setting the maximum steps to 3 decreases around 1.0 success rate in validation data. We set the maximum response length to 512. We set the temperature to 0.5 for sampling trajectories during training and use greedy decoding during inference.

A.4. Cursor-Centric Focusing

During training, we set the maximum resolution P to 1920×1080 , and any larger image will be downsampled to this resolution while keeping the original aspect ratio. Qwen2.5-VL-7B uses a patch size of 14×14 and aggregates each 4 adjacent patch features before forwarding to the transformer, so the maximum number of tokens for each image is around 26k. During inference, we apply CCF to obtain better accuracy in the task with a higher resolution than the maximum resolution during training. It crops the full image based on the initial prediction. The crop is sized to the maximum training resolution while maintaining the original image’s aspect ratio. It attempts to centre this crop on the initial prediction; however, if the prediction is near an edge, the crop area is shifted to ensure it remains entirely within the image boundaries. Though we have shown that CCF can improve the accuracy effectively, its accuracy could be impacted by the initial prediction, because the target could be out of the focused area when the initial prediction is far away from the target; we find that 10.3% of examples in ScreenSpot-Pro have this issue. Several potential strategies could be used to alleviate this issue, such as increasing the training resolution size to improve the initial prediction precision, keeping the initial image in the interaction history,

Model	Mobile		Desktop		Web		Average
	Text	Icon/Widget	Text	Icon/Widget	Text	Icon/Widget	
GPT-4o	20.5	22.7	21.1	24.3	10.0	6.3	17.5
+ Direct Move, 10 Steps	29.3	30.1	26.3	20.0	14.8	9.7	21.7
+ Relative Move, 10 Steps	46.3	27.7	36.6	15.0	17.5	9.9	25.5
Qwen-2.5-VL-7B	97.6	87.2	90.2	74.2	93.2	81.3	88.8
+ Direct Move, 4 Steps	50.0	28.3	57.2	19.0	42.9	20.3	36.3
+ Relative Move, 4 Steps	2.7	2.1	1.7	1.6	0.0	0.0	1.3

Table 6. ScreenSpot-v2 accuracy (%) of VLMs without fine-tuning using the direct and relative moving strategies.

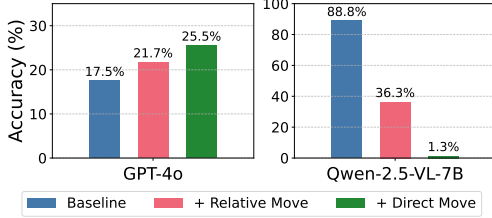


Figure 7. ScreenSpot-v2 accuracy in different moving strategies.

and training the model to decide which area to focus. We will explore these strategies in future work.

B. Relative and Direct Moving Strategies

We try two action strategies for moving the cursor: *relative move* and *direct move*. In the relative move strategy, the model outputs relative offset $(\Delta x, \Delta y)$, and the position of the cursor will be moved from (x, y) to $(x + \Delta x, y + \Delta y)$, where Δx and Δy are integers, positive and negative Δx moves the cursor right and left, and positive and negative Δy moves the cursor down and up, respectively. We evaluate both strategies for GPT-4o and Qwen-2.5-VL-7B. We present the prompt used by GPT-4o in Figure 22.

We tuned the prompts of Qwen-2.5-VL-7B for both strategies. However, we found that it fails to conduct relative movement with an accuracy close to zero. It might be because the model has been heavily fine-tuned on grounding data that requires directly outputting the coordinates. Therefore, in GUI-CURSOR, we use the direct move strategy with better starting accuracy for RL training. The system prompt used by GUI-CURSOR is presented in Figure 21.

C. Movement Steps and Target Sizes

We analyse the relationship between the target UI element size and the moving steps. In Figure 9, we group the samples into two groups: only conducting one step movement after CCF, and conducting more than one step movement after CCF. Within each group, we then calculate the average target size and the average proportion of the target in the image, shown in the first and second row of Figure 9, respectively.



Figure 8. Average movement steps and reward in during training.

The first row shows that the samples with more than one step movement have a smaller target size. The second row further shows that the target with a smaller relative size in an image may need more movement steps. These results show that the smaller targets may increase the difficulty of the tasks, and thus the model may conduct more movement steps in these samples. We also analyse the average number of image pixels in each group, and we found that the higher resolution may not always be related to more movement steps, e.g., in ScreenSpot-Pro, the average number of pixels in the moving one-step group is 56×10^5 , which is larger than the moving more steps group 50×10^5 .

D. Training Dynamics

We present the average number of response tokens, trajectory reward R_T (Equation (6)), and the success rate in validation data during training in Figure 10, where the token numbers and the trajectory reward are calculated after the online filtering. According to Figure 10 and Figure 8, we observe that the learning of GUI-CURSOR mainly has three stages.

Phase one (cold start) – From 0 to 25 steps, the response length decreases, and the trajectory reward increases. At this stage, the model learns to output the correct format, and the accuracy of prediction improves, resulting in higher trajectory reward values.

Phase two (single-step grounding) – From 50 to 150 steps, the response length decreases, and the trajectory reward

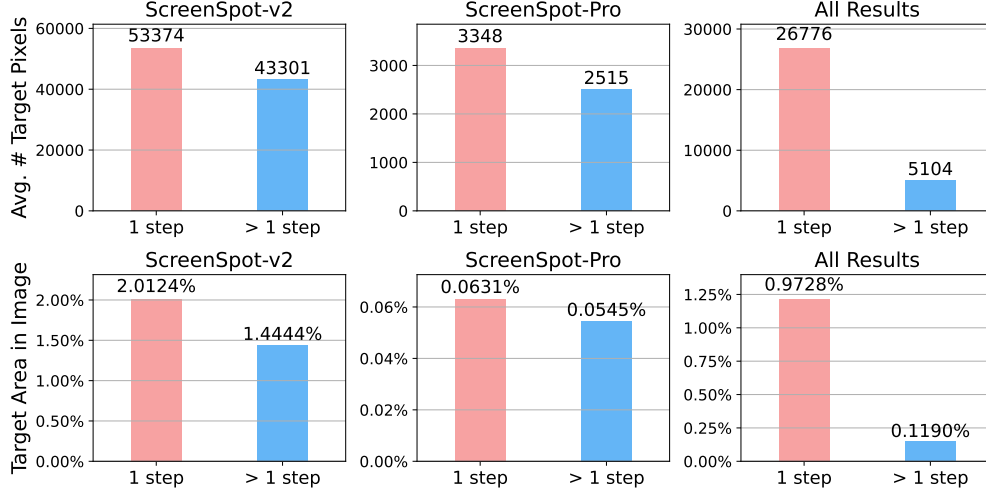


Figure 9. Analysis of the relationship between moving steps and the size of the target UI element. Above: the average number of pixels of the targets. Below: the average proportion of target size in the GUI image. All results in the last column refer to the average values in both datasets. The red and blue bars refer to the samples that move one step and more than one step, respectively. The comparisons show that the target size is often smaller in the samples that move more than one step.



Figure 10. The training dynamics of GUI-CURSOR. The first and the second figures show the average number of response tokens and the average trajectory reward in each batch during training, respectively; the values are calculated after the online filtering. The third figure shows the success rate every 20 training steps in the validation data; the success rate metric requires the final prediction within the target area, and the model explicitly outputs STOP at the last step.

also decreases. Since we apply the online filtering, the easy training samples will be filtered out. The decrease in the trajectory reward indicates that more difficult samples are used to train the model. As shown in Figure 8, the average number of movement steps is close to 1 at this stage. These results indicate that at this stage, the model continues to improve one-step grounding accuracy but hasn’t learnt a good cursor moving policy.

Phase three (multi-step movements) – From 150 to 200 steps, the trajectory rewards, the response length, and the average number of movement steps increase. This indicates that the model learns to execute multiple movements to refine the cursor position, thereby obtaining better accuracy. Additionally, the increase in response length may indicate that the model generates more spatial analysis, thereby improving the moving correctness.

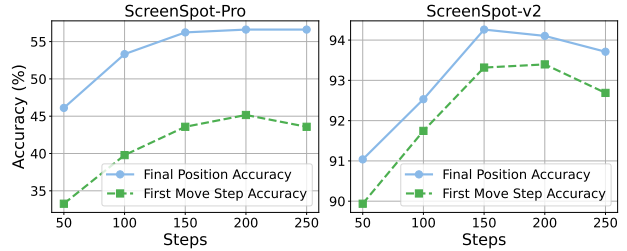


Figure 11. Accuracy of intermediate checkpoints on test sets.

We also present the evaluation results on saved checkpoints (every 50 steps) in Figure 11, which also shows a three-stage learning property. We observe that on ScreenSpot-v2, the accuracy is highest at the 150th step, which is the end of the second stage. In contrast, on the more difficult ScreenSpot-Pro benchmark, the accuracy continues to improve after

the 150th steps. These results indicate that 1) the model improves one-step grounding accuracy at the first two stages, which is usually sufficient for easier tasks in ScreenSpot-v2; 2) it learns rational multi-step movement at the third stage, which helps to improve accuracy for more difficult tasks in ScreenSpot-Pro; and 3) though it hasn’t learn a moving policy well at the second stage, the visual feedback helps the alignment between coordinates and their on-screen positions, and the accuracy at the 150th steps in ScreenSpot-v2 is 94.2, significantly higher than the baselines presented in Table 3.

E. Cursor-in-Box Test

We design the ”cursor-in-box” test to evaluate how well a VLM can infer the spatial relationship between a cursor and another object. We present the model with a white background showing a red bounding box and a black mouse cursor, and the model classifies whether the cursor is inside the box. We present the prompt in Figure 20. We generate the test dataset by putting the bounding box across different positions of the white background, and putting the cursor inside or outside each box. In Figure 12, we present an example showing a box in a white background, and several cursors around it are used to test the classification accuracy at this box position. We only show one box with one cursor for each test example. For each box, we select 5 positions inside it, with 4 close to the corners and one at the centre; the positions outside the box have a distance shorter than 3 times the cursor size. We test the classification F1 scores at different positions of the background, and then obtain the heatmap as shown in Figure 6.

F. Training and Inference Efficiency

We compare the number of training samples of different grounding models in Table 7. We can see that GUI-CURSOR achieves the best accuracy while using the fewest training samples, demonstrating the data efficiency feature of our method. Regarding the efficiency of multi-turn RL, the average training time per step for one-step and four-step movements is 289 and 680 seconds in our implementation. The primary bottleneck is online trajectory rollout, which increases from 157 to 432 seconds when using four-step movement in our implementation. To accelerate the multi-step rollout, we can improve parallelism through asynchronous processing or by adding more machines for the rollout. In summary, our method remains more efficient overall, achieving better accuracy with significantly fewer samples and fewer total steps.

In Table 8, we compare the inference speed when taking different steps of movement. Compared to one-step inference, the samples that conduct 2- and 3-step movement

Model	# Training Samples	ScreenSpot-Pro Acc.
<i>Initialised from Qwen2.5-VL-7B</i>		
Qwen2.5-VL-7B (Bai et al., 2025)	-	26.8
GUI-Actor-7B (Wu et al., 2025b)	1.76M	47.7
GUI-G ² -7B (Tang et al., 2025a)	100K	47.5
GUI-Spotlight-7B (Lei et al., 2025)	18.6K	38.7
GUI-Cursor-7B	8K	56.5
<i>Initialised from UI-TARS-1.5-7B</i>		
UI-TARS-1.5-7B (Qin et al., 2025)	-	38.7
GTA1-7B (Yang et al., 2025a)	64K	50.1
GUI-Spotlight-7B (Lei et al., 2025)	18.6K	52.8
GUI-Cursor-7B	8K	58.1

Table 7. Comparison of the number of training samples and the accuracy on ScreenSpot-Pro.

Steps	1	2	3
# Samples per Second	2.35	2.08	1.38

Table 8. Inference speed (Samples per Second) by number of steps.

decrease the throughput by 11% and 41%. As 95% of samples can be solved within two steps, our method decreases the efficiency by 12.5% compared to one-step methods on average. Beyond the throughput analysis in Table 8, we report the end-to-end wall-clock latency of GUI-CURSOR and compare it against the one-step baseline GTA1-7B (Yang et al., 2025a). The latency includes the cursor rendering time and the input-encoding cost of the interaction history. On ScreenSpot-v2, where most queries can be solved in a single step, GUI-CURSOR adds only ~ 0.1 s of latency over GTA1-7B for a marginal accuracy gain. On the more challenging ScreenSpot-Pro, full multi-step inference increases per-sample latency by ~ 0.3 s but yields a 6.4% accuracy improvement over GTA1-7B. The latency–accuracy trade-off therefore strongly favours GUI-CURSOR on high-resolution, complex GUI screenshots. In future work, the maximum number of movement steps could be set adaptively based on the difficulty of each task (e.g., the planning model could estimate the difficulty from the screenshot resolution and the approximate target size) to further reduce average latency without sacrificing accuracy on hard examples.

G. Concurrent Iterative Cropping Methods

Some concurrent works propose to crop GUI images and narrow the prediction region iteratively (Lei et al., 2025; Ye et al., 2025; Du et al., 2025). We compare CCF with them in the following. GUI-Spotlight (Lei et al., 2025) introduces three tools for narrowing the region iteratively and is trained by multi-step RL. CCF is simpler and more effective, e.g., applying CCF to GTA1 surpasses GUI-Spotlight on ScreenSpot-Pro by 2.2% without training. GUI-ARP (Ye et al., 2025) proposes a cropping strategy by taking the signals from attention scores, but it needs two-stage large-scale training. Though it shows advantage on the larger images, it is less effective in tasks with standard image sizes (e.g.,

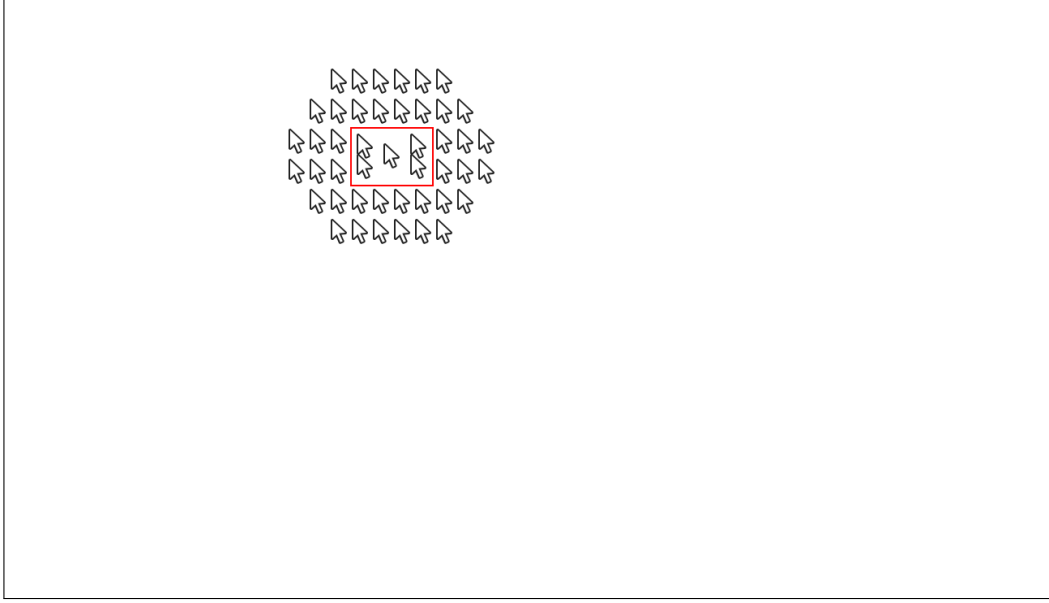


Figure 12. Example of a box position in the white background. The black frames are used for edge visualisation and are not a part of the image for testing. For each test example, we only show one box with one cursor in the white background.

Model	ScreenSpot-v2		ScreenSpot-Pro	
	Acc. (%)	Latency (s)	Acc. (%)	Latency (s)
GTA1-7B, 1 step	92.4	0.26	50.1	0.58
GUI-Cursor-7B (Qwen), 1 step	93.2	0.27	45.3	0.59
GUI-Cursor-7B (Qwen), 2 steps	93.8	0.37	56.0	0.71
GUI-Cursor-7B (Qwen), full (max 4)	93.9	0.37	56.5	0.89

Table 9. End-to-end wall-clock latency (Seconds per Sample) and accuracy of GTA1-7B (single-step baseline) and GUI-CURSOR under different maximum movement-step budgets.

Model	ScreenSpot-Pro	Max Steps	Latency
GTA1-7B (UI-TARS-1.5-7B)	50.1	1	0.58
GUI-CURSOR-7B (Qwen2.5-VL-7B)	56.5	4	0.89
RegionFocus (Qwen2.5-VL-7B)	32.1	100	5.7
RegionFocus (UI-TARS-7B)	41.2	100	-
RegionFocus (UI-TARS-7B)	~45.0	300	-

Table 10. Comparison with the test-time scaling method RegionFocus on ScreenSpot-Pro. GUI-CURSOR achieves substantially higher accuracy with far fewer inference steps and lower latency.

it only obtains 91.8% accuracy in ScreenSpot-v2) due to it does not solve the spatial semantic alignment issue. GUI-RC/RCPO (Du et al., 2025) samples multiple predictions and then conducts voting or test-time RL to improve the grounding accuracy. GUI-RC also does not require training, but CCF remains simpler and more effective, e.g., in ScreenSpot-Pro, GUI-G2-7B +CCF achieves 52.1% accuracy, but GUI-G2-7B +GUI-RC only achieves 47.9%.

H. Compare with Test-Time Scaling Methods

We further compare GUI-CURSOR against the recent test-time scaling approach RegionFocus (Luo et al., 2025b), which iteratively narrows the region of interest by predicting multiple candidate areas at inference time. We evaluate RegionFocus using its officially released code on the same hardware as our experiments. Table 10 summarises the comparison. RegionFocus requires up to 100 inference steps per sample on Qwen2.5-VL-7B, yielding an average latency of 5.7 s, roughly $10\times$ slower than the one-step baseline. Despite this much larger compute budget, RegionFocus reaches only 32.1% accuracy on ScreenSpot-Pro with Qwen2.5-VL-

7B. In contrast, GUI-CURSOR attains 56.5% in at most 4 steps and with an average latency of 0.89 s, a 24.4% accuracy gain at over $6\times$ lower latency.

I. Compare with Qwen3-VL Backbones

To assess whether the gains of GUI-CURSOR persist on more recent and stronger backbones, we train an additional model, GUI-CURSOR-2B, initialised from Qwen3-VL-2B-Thinking (Qwen Team, 2025), and compare against the off-the-shelf Qwen3-VL series at three scales (2B, 8B, and 32B). We adopt the same training data and hyperparameters used for the main 7B experiments. Table 11 summarises the results across ScreenSpot-v2, ScreenSpot-Pro, and OSWorld-G. GUI-CURSOR-2B improves Qwen3-VL-2B-Thinking from 32.2% to 55.0% on ScreenSpot-Pro, surpassing the much larger Qwen3-VL-8B-Thinking (46.6%). Even the 2B variant trained with GUI-CURSOR approaches the larger Qwen3-VL-32B-Thinking on ScreenSpot-Pro (55.0% vs. 57.1%). These results indicate that the interactive cursor-moving paradigm is complementary to and benefits from stronger pre-trained VLMs.

Model	ScreenSpot-v2	ScreenSpot-Pro	OSWorld-G
Qwen3-VL-32B-Thinking (Qwen Team, 2025)	95.7	57.1	64.0
Qwen3-VL-8B-Thinking (Qwen Team, 2025)	93.6	46.6	56.7
GUI-CURSOR-7B (Qwen2.5-VL-7B)	93.9	58.1	65.6
Qwen3-VL-2B-Thinking (Qwen Team, 2025)	88.9	32.2	41.8
GUI-CURSOR-2B (Qwen3-VL-2B-Thinking)	92.6	55.0	58.5

Table 11. Accuracy (%) when applying GUI-CURSOR to recent Qwen3-VL-Thinking backbones. GUI-CURSOR-2B outperforms the off-the-shelf Qwen3-VL-8B-Thinking on ScreenSpot-Pro and OSWorld-G.

J. Recovering from Out-of-Crop Predictions

As noted in Section 2.4, when the initial coarse prediction is far from the target ($\sim 10.3\%$ of cases on ScreenSpot-Pro), the target may be cropped out of the focused view and GUI-CURSOR has no built-in mechanism to “zoom out” or re-centre. We conduct a preliminary experiment to assess whether GUI-CURSOR already possesses the zero-shot capability to detect this failure mode. Using the cropped views from ScreenSpot-Pro, we prompt GUI-CURSOR-7B with a binary classification task: “Given a GUI screenshot and a user instruction, determine whether the target element described in the instruction is within the screenshot. Answer with ‘Yes’ if the target element is within the screenshot, and ‘No’ if it is not. Instruction: {instruction}.” The model correctly identifies that the target is missing in 31% of cases where the target was indeed cropped out, but it also incorrectly reports the target as missing in 20% of cases where the target was actually present. Because the base rate of out-of-crop predictions is low ($\sim 10.3\%$), a 20% false-positive rate would cause the system to abandon many correct crops and degrade overall accuracy. Implementing a reliable zoom-out / re-centre action therefore likely requires explicitly incorporating a target-presence classification signal into the reward function during training, which we leave to future work.

K. Cursor Occlusion of Small Targets

A potential concern with rendering a virtual cursor onto the GUI image is that the cursor may occlude very small UI elements (e.g., a 10×10 -pixel close button) at the final step, hindering fine-grained verification. GUI-CURSOR mitigates this by conditioning each step on the entire interaction history $(I, O_0, A_0, \dots, A_{t-1}, O_t)$, which provides a form of sequential *visual memory*. The model can therefore reason about the target’s appearance from earlier observations (in which the cursor was elsewhere) and use that context to confirm that a partially occluded cursor is correctly positioned on the target. Figure 18 in Appendix M provides a qualitative example: the target is very small and partially occluded by the cursor at the last step, yet the model correctly infers the cursor lies on the target and issues STOP using context from the prior observations.

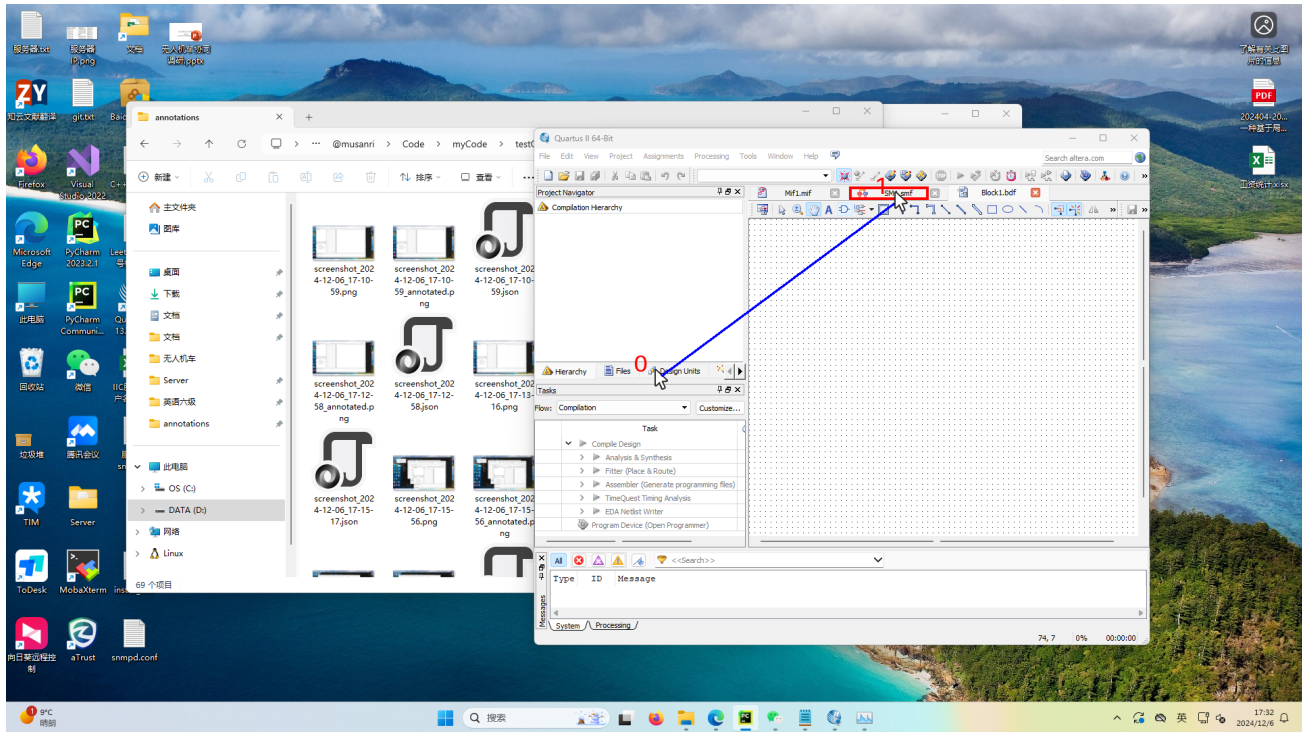
L. Extension to Continuous GUI Interactions

GUI-CURSOR focuses on click-style grounding, which is the dominant setting in current GUI grounding benchmarks. Continuous interactions such as dragging or sliding require predicting two coordinates (a start point $(x_{\text{start}}, y_{\text{start}})$ and an end point $(x_{\text{end}}, y_{\text{end}})$) and are largely under-explored. The recent work GUI-Drag (Liao et al., 2026) formulates drag-like actions as a two-coordinate prediction task and shows that current grounding models struggle on it, e.g., Qwen2.5-VL-7B recognises only 5% of drag tasks and achieves a 2.6% success rate; but fine-tuning on drag-style data substantially closes the gap. GUI-CURSOR can be extended to such interactions by letting the high-level planning model sequentially invoke the grounding model to predict the two coordinates: first the cursor is moved to $(x_{\text{start}}, y_{\text{start}})$ using the interactive search introduced in Section 2.2, then a second invocation predicts $(x_{\text{end}}, y_{\text{end}})$. The main requirement for this extension is the availability of drag-style training data so that the model can learn the cross-coordinate dependencies; the interactive feedback loop and the trajectory-based reward used by GUI-CURSOR transfer naturally to each individual coordinate prediction. We leave a full empirical evaluation of continuous interactions to future work.

M. Case Study

We present case studies and visualise the prediction process of GUI-CURSOR. We render the predicted cursor trajectory as a sequence of positions connected by blue lines. Each cursor position is labelled with a red number at its top-left position to indicate the step number. The target is highlighted with a red bounding box. If CCF is applied, the corresponding focused region is shown with an orange bounding box.

We present positive examples with single-step movement (Figure 13), multi-step movement (Figure 19, Figure 14 and Figure 18). We also present negative examples with single-step movement (Figure 17), and multi-step movement (Figure 15 and Figure 16).

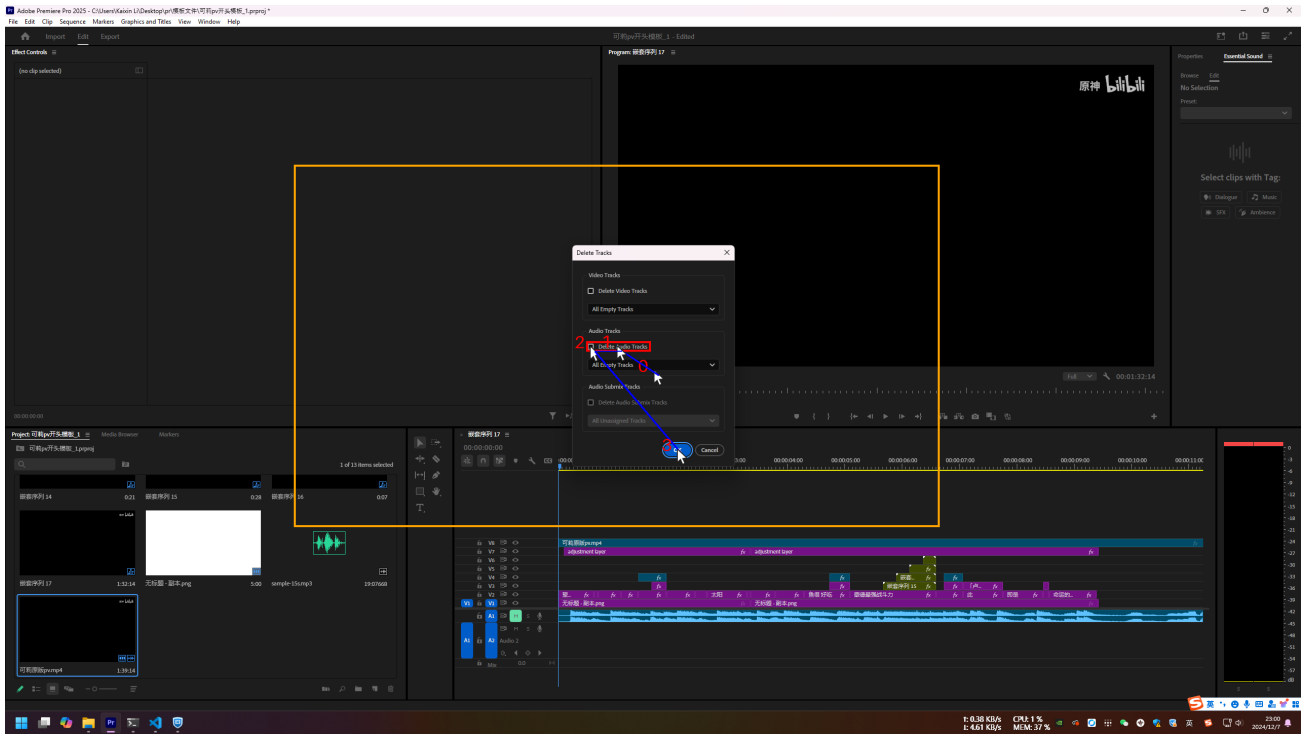


Task: select the SM1.smf file in Quartus window

Step 0: *<think>*To select the "SM1.smf" file in the Quartus window, I need to click on the tab or button that represents this file. It appears to be located near the top of the Quartus window, among other tabs.*</think><answer>(1324, 281)</answer>*

Step 1: *<think>*The target, the "SM1.smf" file, has been successfully located and the cursor is positioned over it, indicating the task is complete.*</think><answer>STOP</answer>*

Figure 13. Positive example from ScreenSpot-Pro with single-step movements.



Task: i want to delete all blank audio tracks

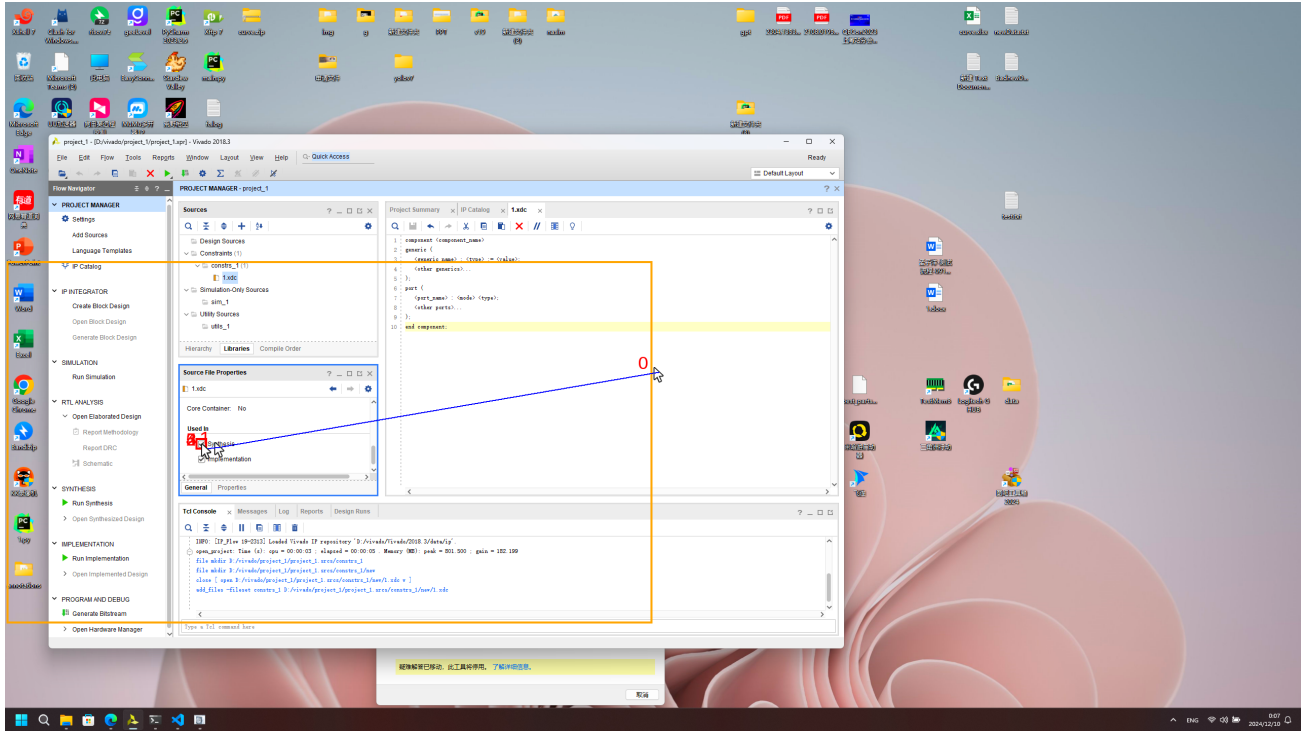
Step 0: <think>To delete all blank audio tracks, I need to click on the "Delete Audio Tracks" checkbox and select "All Empty Tracks" from the dropdown menu.</think><answer>(1202, 667)</answer>

Step 1: <think>To delete all blank audio tracks, I need to click on the checkbox labeled "Delete Audio Tracks" and ensure the dropdown is set to "All Empty Tracks".</think><answer>(784, 478)</answer>

Step 2: <think>The target, the checkbox for "Delete Audio Tracks," appears to be correctly clicked, aligning with the goal to delete all empty audio tracks. The next logical step is to confirm this action by interacting with the "OK" button.</think><answer>(1013, 746)</answer>

Step 3: <think>The goal is to confirm the deletion of all empty audio tracks. The "OK" button is clearly visible and seems to be the appropriate button to click to confirm the action.</think><answer>STOP</answer>

Figure 15. Negative example from ScreenSpot-Pro with multi-step movements. The task is to locate the position for the given instruction, but the model hallucinates taking a series of actions.



Task: turn off the synthesis of 1.xdc in vivado

Step 0: <think>To turn off the synthesis of 1.xdc in Vivado, I need to uncheck the "Synthesis" checkbox in the "Source File Properties" window.</think><answer>(412, 861)</answer>

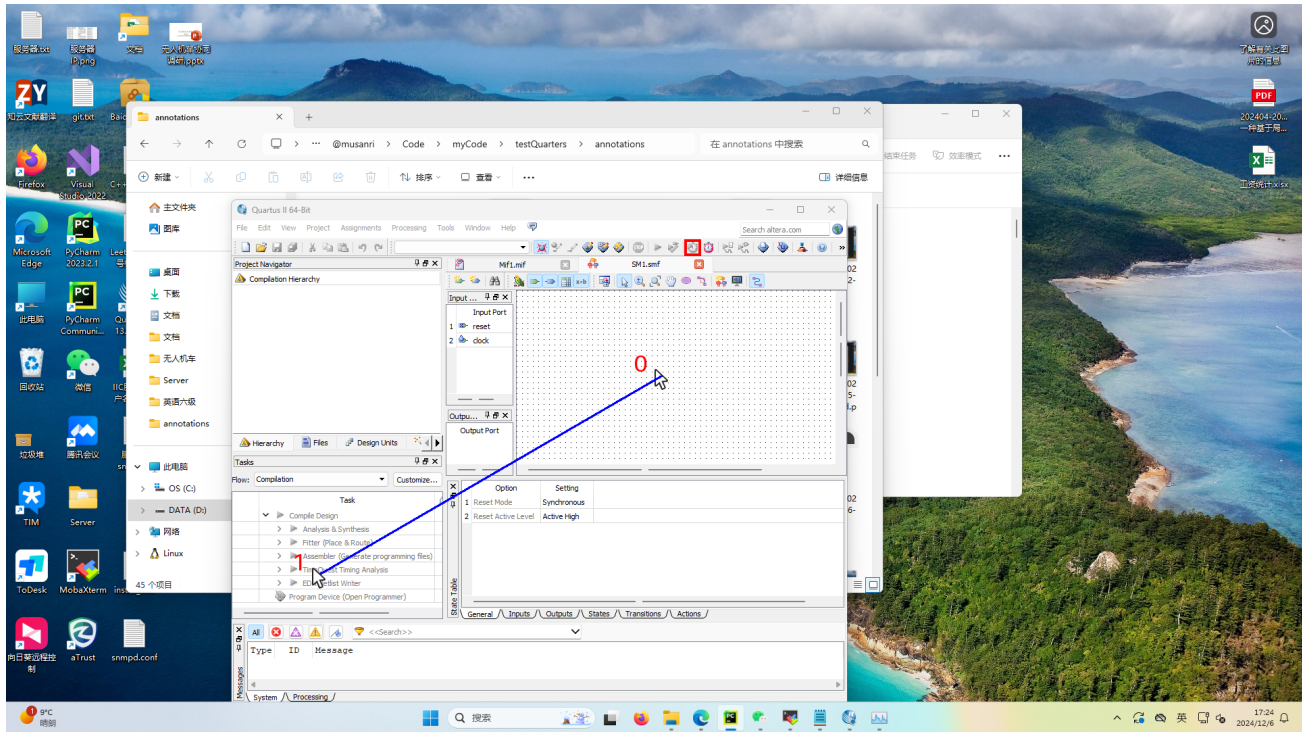
Step 1: <think>To turn off the synthesis of the 1.xdc file, I need to uncheck the "Synthesis" checkbox in the "Source File Properties" window.</think><answer>(517, 480)</answer>

Step 2: <think>The task is to uncheck the "Synthesis" option in the "Source File Properties" window. The checkbox for "Synthesis" is currently checked, and it needs to be unchecked to turn off synthesis.</think><answer>(517, 480)</answer>

Step 3: <think>To turn off the synthesis of the 1.xdc file, I need to click on the checkbox labeled "Synthesis" to uncheck it. This will disable the synthesis for the file.</think><answer>(517, 480)</answer>

Step 4: <think>I need to uncheck the "Synthesis" option in the "Source File Properties" window to turn off the synthesis for the 1.xdc file. The checkbox for "Synthesis" is currently checked, and clicking on it should toggle the checkbox off.</think><answer>(517, 480)</answer>

Figure 16. The cursor is correctly positioned, but the same position is predicted repeatedly. The model is waiting for the action to be executed.

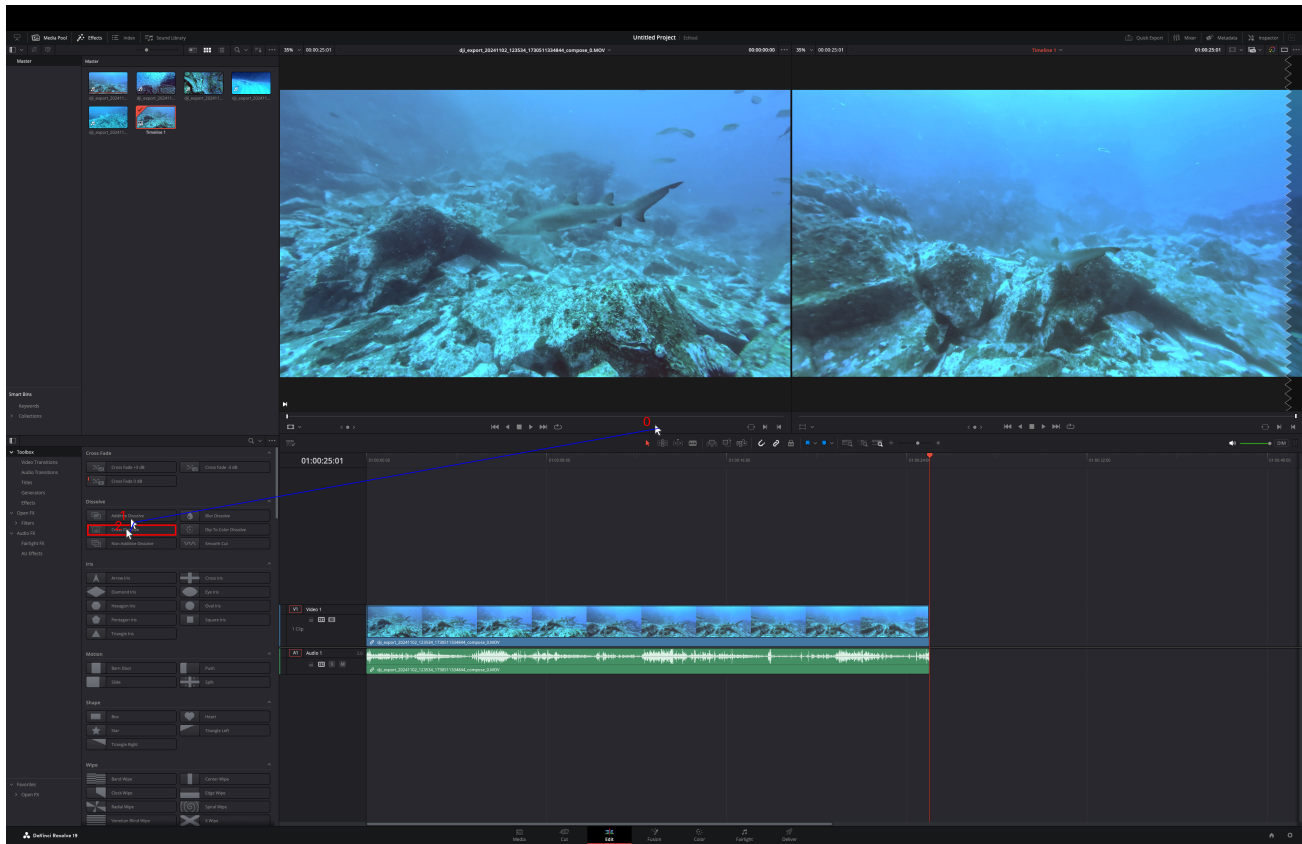


Task: Start TimeQuest Timing Analyzer in toolbar of Quartus

Step 0: <think>To start the TimeQuest Timing Analyzer in the toolbar of Quartus, I need to locate and click on the corresponding icon in the toolbar area of the Quartus window.</think><answer>(456, 844)</answer>

Step 1: <think>The target, the TimeQuest Timing Analyzer button, appears to be correctly identified and clicked in the previous step.</think><answer>STOP</answer>

Figure 17. Negative example of one-step movement. The model misidentifies the target



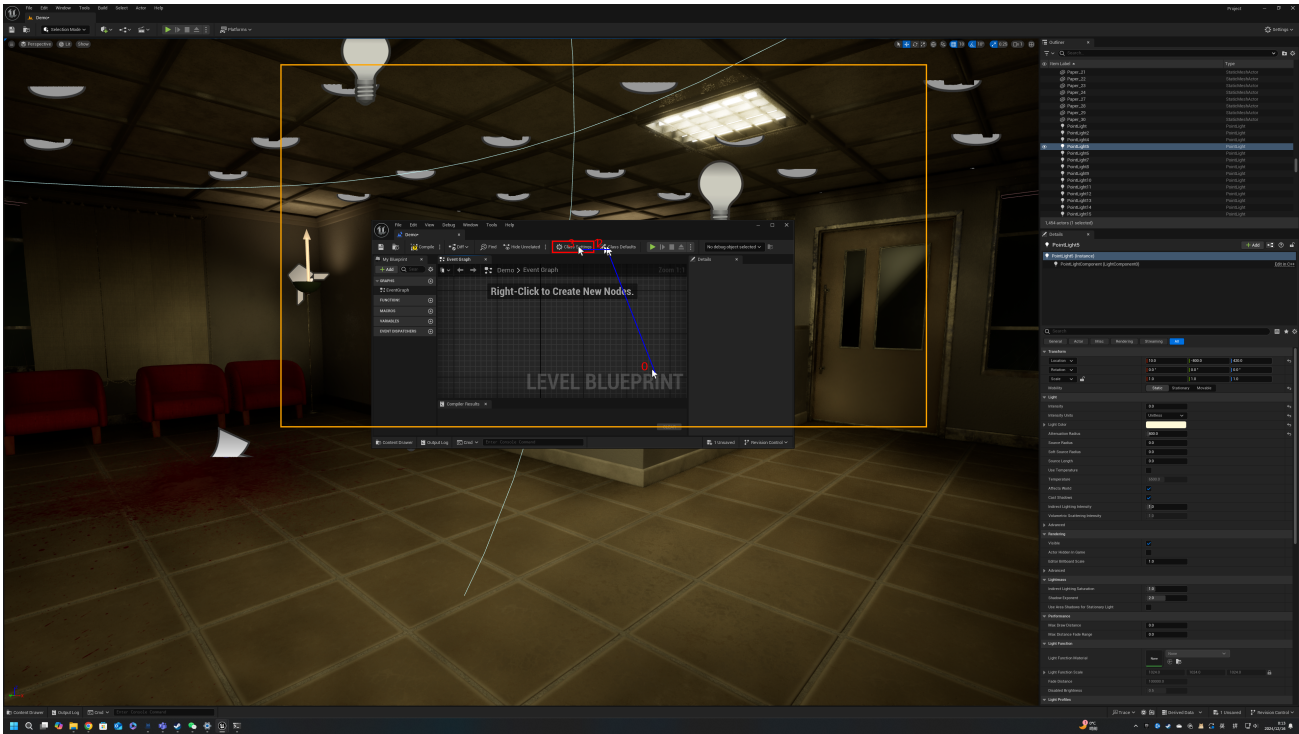
Task: Cross Dissolve

Step 0: <think>To locate the "Cross Dissolve" option, I need to find the icon and label associated with it in the toolbox. It appears to be a circular icon with a red border, which is typically used to represent a dissolve effect.</think><answer>(330, 1371)</answer>

Step 1: <think>Given the new position, it seems the "Cross Dissolve" is correctly identified.</think><answer>(318, 1398)</answer>

Step 2: <answer>STOP</answer>

Figure 18. Positive example with multi-step movement. The model outputs STOP at the last step without thinking.



Task: Edit Class Settings

Step 0: <think>To locate the "Edit Class Settings" button, I need to scan the area near the top of the interface where such UI elements are typically placed. It appears to be in the vicinity of the "Class Settings" label.</think><answer>(1738, 696)</answer>

Step 1: <think>To locate the "Edit Class Settings" button, I need to look for a button that typically represents settings or configurations, often depicted with a gear icon or a settings gear symbol. It is usually positioned near the top of the interface, often in a toolbar or menu.</think><answer>(975, 548)</answer>

Step 2: <think>The target, the "Edit Class Settings" button, seems to be misidentified. It appears that the "Class Defaults" button is being pointed at instead. The "Edit Class Settings" button is likely the one with a gear icon, which is typically used for settings, and it is located to the left of the "Class Defaults" button.</think><answer>(890, 548)</answer>

Step 3: <think>The goal is to click on the "Edit Class Settings" button, which is part of the toolbar and visually distinct from the "Class Defaults" button. The "Class Settings" button is the one with a gear icon, and it is correctly identified in the previous step.</think><answer>STOP</answer>

Figure 19. Positive example with multi-step movement.

You are given an image showing a white background, a red bounding box, and a computer mouse cursor. Is the cursor located inside the red bounding box? Please answer with "Yes" or "No".

Figure 20. The prompt of "cursor-in-box" test.

You are an expert UI element locator. Given a GUI image and a user's element description, your goal is to provide the coordinate of the element intended by the user. A cursor will be exactly placed to the coordinate provided by you. Please make sure to check whether the cursor is at the target position.

The image resolution is height {height} and width {width}. The cursor is a black arrow with white fill, initialized at the center of the image. The cursor's hotspot is at the top-left corner.

Please output `<answer>STOP</answer>` if the cursor's hotspot is on the target position. Otherwise, provide a new coordinate by `<answer>(x, y)</answer>`, where x and y must be positive integer values. You will receive a new image with updated cursor position at each turn.

Your response must contain a thinking process before the answer. The thinking process is enclosed in a `<think>` tag, and the answer is enclosed in an `<answer>` tag. In your thinking process, you should identify the target element and the spatial relation between the cursor and the target. Make sure to use the updated cursor position to refine your estimation about the coordinate of the target element.

Figure 21. The system prompt of GUI-CURSOR.

You are an AI assistant designed for precise cursor control within a graphical user interface.

Your Primary Goal: Based on the user query, accurately move the cursor to the center of the intended target UI element on the screen.

Information Provided at Each Step:

1. **Screenshot:**

- Dimensions: {screen_width} × {screen_height} pixels.
- Content: An image of the current screen, showing the cursor you are controlling.
- Updates: After each MOVE command you issue, a new screenshot will be provided in the subsequent turn reflecting the new cursor position.

2. **Cursor Details:**

- The cursor is black and is initially positioned at the center of the screen.
- If you attempt to move the cursor outside the screen, its position will be automatically adjusted to remain within the screen boundaries.

Your Iterative Task and Output:

1. **Identify Target and Analyze:**

- Carefully examine the user's query and the current screenshot to pinpoint the intended target UI element.
- If your understanding of the target element is incorrect or needs adjustment during the process, revise your identified target.
- Determine the relative position between the cursor's current position and the center of your identified target UI element.

2. **Determine Action & Output:**

- **If the cursor is not yet at the target:**
 - **Output:** MOVE(dx, dy)
 - **dx (Horizontal Movement):**
 - * Positive dx moves the cursor right. Negative dx moves the cursor left.
 - **dy (Vertical Movement):**
 - * Positive dy moves the cursor down. Negative dy moves the cursor up.
- **If the cursor is at the target:**
 - **Output:** STOP
 - This command should only be issued when the cursor is accurately positioned at the center of the intended UI element.

Figure 22. The system prompt of GPT-4o used in the relative move strategy.